sumo_morl

Többcélú megerősítéses tanulás jelzőlámpa-vezérléshez SUMO-ban

Tartalomjegyzék

1. sumo_morl – többcélú megerősítéses tanulás jelzőlámpa-vezérléshez 3
- 1.1. Telepítés . . . 3
- 1.2. Demó: fix idejű program . . . 3
- 1.3. Gyors kezdés . . . 4
- 1.4. Példák . . . 4
- 1.5. A könyvtár felépítése . . . 5

2. A csomópont, a fázisok és a közbenső idők 5
- 2.1. A csomópont . . . 5
- 2.2. Fogalmak . . . 6
- 2.3. A közbenső idők számítása . . . 6
- 2.4. A fázisok . . . 7
- 2.5. Az átmenetek . . . 7
- 2.6. Kiírás és ábrák a saját gépeden . . . 7

3. Megfigyelés: hurokdetektorok (csak mérhető adatok) 8
- 3.1. A detektorok elhelyezése . . . 8
- 3.2. Mit mér egy hurok? . . . 8
  - 3.2.1. Becsült járműszám ágankként (bemenet–kimenet módszer) . . . 8
- 3.3. A megfigyelésvektor . . . 8
- 3.4. A nyers detektoradatok: `info["detectors"]` . . . 9
- 3.5. Amit szándékosan NEM kap meg az ágens . . . 9

4. Jutalomvektor: célfüggvények 9
- 4.1. Beépített célfüggvények . . . 10
- 4.2. Saját célfüggvény . . . 10
- 4.3. Skalarizálás . . . 10

5. A környezet API-ja 11
- 5.1. Akció . . . 11
- 5.2. Lépés és epizód . . . 11
- 5.3. Az `info` szótár . . . 12
- 5.4. Forgalmi igény . . . 12
- 5.5. Megjelenítés . . . 12

6. MORL segédfüggvények és összehasonlító vezérlők 12
- 6.1. Epizód futtatása, kiértékelés . . . 12
- 6.2. Skalarizálás és súlyok . . . 13
- 6.3. Pareto-front és hipertérfogat . . . 13
- 6.4. Ábrák . . . 13
- 6.5. Összehasonlító vezérlők (baseline-ok) . . . 13
- 6.6. Párhuzamos környezetek . . . 14

6.7. Sebesség . . . 14

7. Feladatötletek hallgatóknak **14**
7.1. Bevezető . . . 14
7.2. Egycélú → többcélú . . . 14
7.3. Haladó . . . 15
7.4. Kiértékelési tanácsok . . . 15

A. A teljes forráskód **16**
A.1. Csomag belépési pont: `sumo_morl/__init__.py` . . . 16
A.2. A környezet: `sumo_morl/env.py` . . . 17
A.3. Célfüggvények: `sumo_morl/objectives.py` . . . 23
A.4. Hurokdetektorok: `sumo_morl/detectors.py` . . . 25
A.5. Jelzésterv a közbenső időkből: `sumo_morl/intergreen.py` . . . 28
A.6. Forgatókönyv: `sumo_morl/scenario.py` . . . 33
A.7. Hálózat generálása: `sumo_morl/network.py` . . . 34
A.8. Forgalmi igény: `sumo_morl/traffic.py` . . . 36
A.9. Összehasonlító vezérlők: `sumo_morl/policies.py` . . . 38
A.10.MORL segédfüggvények: `sumo_morl/morl.py` . . . 40
A.11.Ábrák: `sumo_morl/visualize.py` . . . 44
A.12.SUMO segédfüggvények: `sumo_morl/sumo_utils.py` . . . 47
A.13.Közbensőidő-számítás: geometria: `geometry_utils.py` . . . 48
A.14.Közbensőidő-számítás: hálózat beolvasása: `sumo_parser.py` . . . 54
A.15.Demó: fix idejű program: `examples/sumo_morl/00_demo_fix_ido.py` . . . 55
A.16.Példa: ismerkedés: `examples/sumo_morl/01_ismerkedes.py` . . . 56
A.17.Példa: baseline-ok: `examples/sumo_morl/02_baseline_osszehasonlitas.py` . . . 57
A.18.Példa: súlyozott tanítás (SB3): `examples/sumo_morl/03_sb3_sulyozott_tanitas.py` 58
A.19.Példa: saját célfüggvény: `examples/sumo_morl/04_sajat_celfuggveny.py` . . . 59
A.20.Tesztek: jelzésterv: `sumo_morl/tests/test_plan.py` . . . 60
A.21.Tesztek: környezet: `sumo_morl/tests/test_env.py` . . . 61
A.22.Telepítési leíró: `pyproject.toml` . . . 62

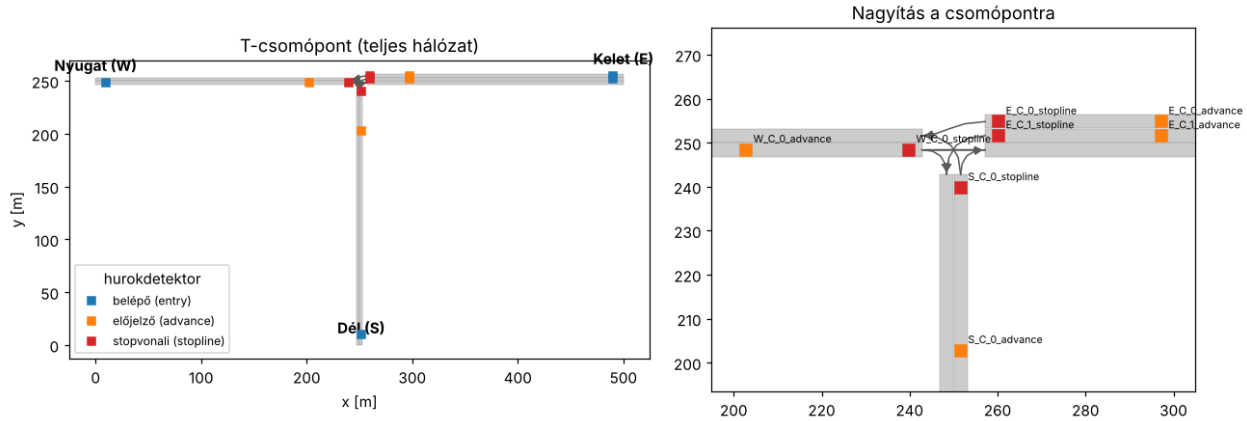
1. sumo_morl – többcélú megerősítéses tanulás jelzőlámpa-vezérléshez

A `sumo_morl` egy oktatási célú Python könyvtár, amellyel **forgalomtechnikai előismeretek nélkül** lehet többcélú megerősítéses tanulással (MORL) jelzőlámpa-vezérlő ágenseket fejleszteni a SUMO forgalomszimulátorban.

Minden forgalomtechnikai rész készen van:

- egy **T-csomópont** hálózata (a keleti ág ketté van osztva balra kanyarodó és egyenes sávra),
- a **közbenső idők** (intergreen) a repó közbensőidő-számító kódjával készülnek, a csomópont geometriájából,
- a **fázisok**, és köztük a szabályos **átmenetek** (sárga, piros, piros-sárga),
- **indukcióshurok-detektorok** minden bejövő sávon,
- **forgalmi igény** generálása (epizódonként véletlen),
- klasszikus **összehasonlító vezérlők** (fix idejű, járműkövető, mohó).

A hallgató feladata „csak” az ágens: minden döntési pontban meg kell mondania, **melyik fázis** legyen a következő.



1.1. Telepítés

Python 3.9 vagy újabb kell. A repó gyökeréből:

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -e .                   # a könyvtár + SUMO (eclipse-sumo pip csomag)
pip install -e ".[rl]"             # + Stable-Baselines3 (a 03-as példához)
```

A SUMO-t az `eclipse-sumo` pip csomag telepíti, így külön SUMO-telepítés nem kell. Ha már van SUMO a gépen és a `SUMO_HOME` környezeti változó be van állítva, azt használja a könyvtár.

Ellenőrzés:

```
python -m pytest                                  # a könyvtár tesztjei (kb. fél perc)
python examples/sumo_morl/00_demo_fix_ido.py --gui # demó: fix idejű program a SUMO felületén
```

1.2. Demó: fix idejű program

Mielőtt bármilyen ágenst írnál, nézd meg a működést a legegyszerűbb, klasszikus vezérléssel: a fázisok mindig ugyanabban a sorrendben, mindig ugyanannyi ideig kapnak zöldet (alapértelmezés: 30, 10 és 15 s).

```
python examples/sumo_morl/00_demo_fix_ido.py --gui                # SUMO grafikus felülettel
python examples/sumo_morl/00_demo_fix_ido.py --green 40 10 20     # más zöldidők
```

A program kiírja a fázisokat, az átmeneteket és a ciklusidőt, majd minden fázisváltásnál a detektorokból becsült járműszámot. A végén megadja a jutalmakat és a valós mutatókat, és elmenti az első két ciklus jelzéstervét (`fix_ido_jelzesterv.png`):



1.3. Gyors kezdés

```python
import numpy as np
from sumo_morl import TJunctionMOEnv

env = TJunctionMOEnv(objectives=("queue", "fairness", "switches"), episode_length=3600)
obs, info = env.reset(seed=0)

done = False
total = np.zeros(env.reward_dim)
while not done:
    action = env.action_space.sample()        # <- ide jön az ágensed
    obs, reward, terminated, truncated, info = env.step(action)
    total += reward                           # a reward egy VEKTOR (célonként egy elem)
    done = terminated or truncated

print(dict(zip(env.objective_names, total)))
env.close()
```

- `action`: a következő fázis sorszáma (0, 1 vagy 2).
- `obs`: 55 elemű vektor [0, 1] tartományban, csak detektoradatokból és a vezérlő saját állapotából.
- `reward`: `numpy` vektor, a kiválasztott célfüggvények sorrendjében.
- `info`: részletes adatok (aktuális fázis, nyers detektoradatok, és – **csak kiértékeléshez** – SUMO-ból kiolvasott valós mutatók).

Egycélú RL könyvtárral (pl. Stable-Baselines3) a jutalmat skalarizálni kell:

```python
from stable_baselines3 import DQN
from sumo_morl import TJunctionMOEnv, LinearScalarization

env = LinearScalarization(TJunctionMOEnv(), weights=[0.6, 0.3, 0.1])
model = DQN("MlpPolicy", env).learn(50_000)
```

A grafikus felülethez add meg a `gui=True` paramétert: ekkor a `sumo-gui` indul el, és látható, mit csinál az ágens.

1.4. Példák

Fájl	Mit mutat be
`examples/sumo_morl/00_demo_fix_ido.py`	**demó:** klasszikus fix idejű program, konzolos eseménynapló, jelzésterv-ábra (`--gui`-val látható a szimuláció)
`examples/sumo_morl/01_ismerkedes.py`	a környezet, a jelzésterv kiírása, véletlen ágens
`examples/sumo_morl/02_baseline_osszehasonlitas.py`	klasszikus vezérlők azonos forgalmon, Pareto-ábra
`examples/sumo_morl/03_sb3_sulyozott_tanitas.py`	DQN tanítása több súlyvektorral, Pareto-front, hipertérfogat
`examples/sumo_morl/04_sajat_celfuggveny.py`	saját célfüggvény definiálása

1.5. A könyvtár felépítése

Modul	Tartalom
`sumo_morl/env.py`	`TJunctionMOEnv` – a Gymnasium környezet
`sumo_morl/objectives.py`	célfüggvények (jutalomkomponensek), `register_objective`
`sumo_morl/detectors.py`	hurokdetektorok elhelyezése, olvasása, aggregálása
`sumo_morl/intergreen.py`	jelzőcsoportok, fázisok, átmenetek a közbensőidő-számításból
`sumo_morl/scenario.py`	a T-csomópont forgatókönyv (hálózat + jelzésterv + detektorok)
`sumo_morl/network.py`	a hálózat generálása `netconvert`-tel
`sumo_morl/traffic.py`	forgalmi igény (`Demand`) és route fájl
`sumo_morl/policies.py`	összehasonlító vezérlők
`sumo_morl/morl.py`	skalarizálás, súlyok, Pareto-front, hipertérfogat, kiértékelés
`sumo_morl/visualize.py`	ábrák (csomópont, fázisok, közbenső idők, epizód)
`sumo_morl/data/t_junction.net.xml`	a kész hálózat

A közbenső időket a repó gyökerében lévő, már meglévő `geometry_utils.py` és `sumo_parser.py` számolja (ugyanaz a kód, amit a GUI-s közbensőidő-számító is használ); a `pip install -e .` ezeket is telepíti.

2. A csomópont, a fázisok és a közbenső idők

> Röviden: **ezzel nem kell foglalkoznod.** Az ágens csak egy fázis sorszámát választja; a környezet gondoskodik róla, hogy a jelzőlámpák mindig szabályosan és biztonságosan váltsanak. Ez az oldal azért van, hogy értsd, mi történik a háttérben.

2.1. A csomópont

Egyszerű T-csomópont, jobbkezes forgalommal. Minden ág 250 m hosszú, a megengedett sebesség 50 km/h.

```
        Nyugat (W) ===============+=============== Kelet (E)
                                   |
                                   |
                                Dél (S)
```

Ág	Bejövő sáv	Mozgások
Nyugat (W)	`W_C_0`	egyenesen keletre + jobbra délre (közös sáv)
Kelet (E)	`E_C_0`	egyenesen nyugatra
Kelet (E)	`E_C_1`	balra, délre (külön kanyarodósáv)
Dél (S)	`S_C_0`	balra nyugatra + jobbra keletre (közös sáv)

A keleti („jobb oldali") bejövő ág tehát **ketté van osztva**: a balra kanyarodók külön sávban várakoznak, így nem tartják fel az egyenesen haladókat.

2.2. Fogalmak

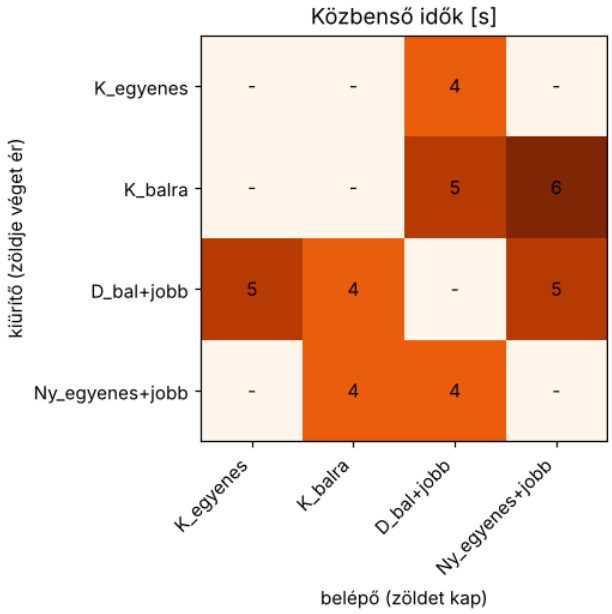
- **Jelzőcsoport**: azon mozgások halmaza, amelyek mindig ugyanazt a jelzést kapják. Nálunk egy bejövő sáv = egy jelzőcsoport (4 csoport).
- **Konfliktus**: két csoport konfliktusos, ha a járműveik útvonala keresztezi egymást vagy összefut. Konfliktusos csoportok nem kaphatnak egyszerre zöldet.
- **Fázis**: konfliktusmentes csoportok halmaza, amelyek egyszerre kapnak zöldet.
- **Közbenső idő** (intergreen) `k(i→j)`: az `i` csoport zöldjének vége és a vele konfliktusos `j` csoport zöldjének kezdete között minimálisan eltelő idő. Ennyi idő kell ahhoz, hogy az utolsó, még zölden behajtó jármű elhagyja az ütközési pontot, mielőtt az új irány járművei odaérnek.
- **Átmenet**: két fázis közötti jelzésképsorozat (sárga, piros, piros-sárga).

2.3. A közbenső idők számítása

A közbenső időket a repó meglévő közbensőidő-számító kódja (`geometry_utils.py`, `sumo_parser.py`) számolja a SUMO hálózat geometriájából:

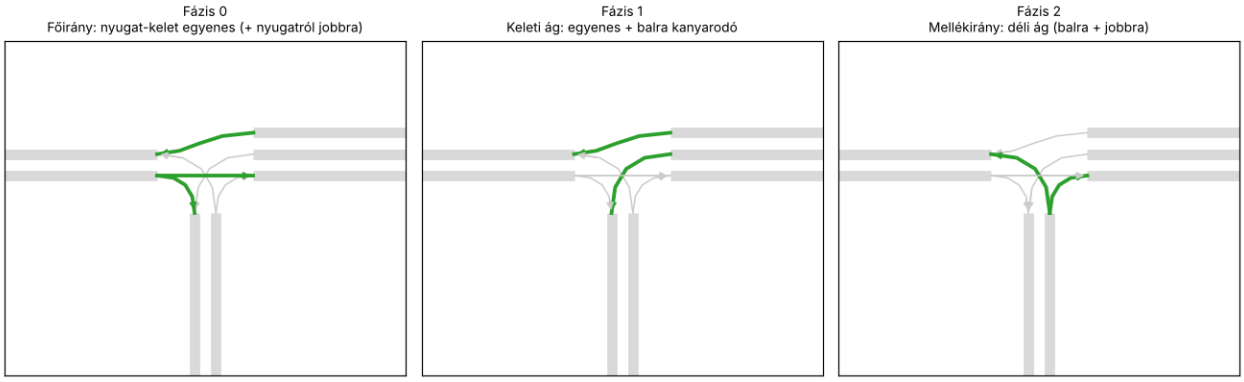
1. minden mozgás pályáját simított görbével közelíti, és kiszámolja a kanyarodási sugarat, abból a sebességet,
2. megkeresi a pályák ütközési pontjait,
3. kiszámolja a kiürítési és a behaladási időt: `k = 3 s + (kiürítési út + 6 m) / v_kiürítő - behaladási út / v_belépő`, felfelé kerekítve.

A `sumo_morl.intergreen` modul ebből a mozgásszintű mátrixból jelzőcsoport-szintű mátrixot képez (a csoport mozgásainak maximuma):



2.4. A fázisok

A fázisokat a konfliktusmátrixból kapjuk: a konfliktusmentes csoportok maximális halmazai (gráfelméletben: klikkek). Ebből a csomópontból pontosan három adódik:



Fázis	Zöldet kap	Megjegyzés
0	`W_C_0` + `E_C_0`	főirány mindkét irányban egyenesen (+ nyugatról jobbra)
1	`E_C_0` + `E_C_1`	keleti ág: egyenes + védett balra kanyarodás
2	`S_C_0`	déli mellékág

2.5. Az átmenetek

Fázisváltáskor a környezet automatikusan lejátssza az átmenetet:

- a megszűnő csoportok 3 s **sárgát**, majd pirosat kapnak,
- az induló csoportok annyi idő után kapnak zöldet, amennyit a közbenső idő előír (legalább 2 s), és előtte 2 s **piros-sárgát** kapnak,
- a mindkét fázisban zöld csoportok végig zöldek maradnak.

Az átmenetek hossza (s):

honnan \ hova	0	1	2
0	–	4	4
1	6	–	5
2	5	5	–

Az átmenetek alatt (és utána a minimális zöldidő alatt) az ágens nem dönt; ez a forgalomtechnikában **elveszett idő**, ezért a túl gyakori váltás rontja az áteresztőképességet.

2.6. Kiírás és ábrák a saját gépeden

```python
from sumo_morl import TJunctionScenario
from sumo_morl.visualize import plot_scenario, plot_phases, plot_intergreen

sc = TJunctionScenario()
print(sc.describe())                # csoportok, közbenső idők, fázisok, átmenetek, detektorok
plot_scenario(sc).savefig("csomopont.png")
plot_phases(sc).savefig("fazisok.png")
```

A hálózat újragenerálása (pl. más ághosszal) csak haladóknak kell: `python -m sumo_morl.network`, illetve `sumo_morl.network.build_network()`.

3. Megfigyelés: hurokdetektorok (csak mérhető adatok)

Egy valódi csomópontban a vezérlő **nem látja** a járműveket: nem tudja, hány autó áll a sorban, ki mióta vár, vagy hová tart. Amit tud, azt az úttestbe épített **indukcióshurok-detektorok** mérik. Hogy az ágens a valóságba is átvihető legyen, a `sumo_morl` megfigyelése és jutalma **kizárólag** ilyen adatokból készül.

3.1. A detektorok elhelyezése

Minden bejövő sávon (4 sáv) három hurok van, összesen 12:

```
ág eleje                                                  stopvonal
|--[entry]--------------------------------[advance]---------[stopline]|
   10 m az ág elejétől                      40 m-re a          3 m-re a
                                            stopvonaltól       stopvonaltól
```

Típus	Mire jó
`entry` (belépő)	a csomópont felé érkező járművek számlálása
`advance` (előjelző)	közeledő járművek érzékelése; ha tartósan foglalt, a sor már idáig ér
`stopline` (stopvonali)	a stopvonalon áthaladó járművek számlálása, a sor eleje

A detektorok azonosítója `<sáv>_<típus>`, pl. `E_C_1_stopline`.

3.2. Mit mér egy hurok?

Mennyiség	Jelentés	Valóságban
`count`	a lépés alatt áthaladt járművek száma	impulzusszámlálás
`occupancy`	az idő hányadában volt jármű a hurok felett (0..1)	foglaltság
`speed`	átlagsebesség a hurkon (m/s), jármű nélkül a megengedett sebesség	dupla hurokkal mérhető
`gap`	a legutóbbi érzékelés óta eltelt idő (s)	időrés, a járműkövető vezérlők alapja

3.2.1. Becsült járműszám ágankként (bemenet–kimenet módszer)

Az `entry` és a `stopline` hurkok számlálóiból folyamatosan számoljuk, hány jármű van a két hurok között: `N(t) = N(t-1) + belépők - kilépők`. Ez a klasszikus, gyakorlatban is használt sorhossz-becslés. Tesztjeink szerint a becslés legfeljebb 1–3 járművel tér el a SUMO-beli valós értéktől.

3.3. A megfigyelésvektor

A `obs` egy 55 elemű, `[0, 1]` tartományba normalizált `float32` vektor:

Elemek	Darab	Normalizálás
minden detektorra: `count`, `occupancy`, `speed`, `gap`	$12 \times 4 = 48$	forgalom / 1800 jármű/óra; foglaltság; sebesség / sebességkorlát; időrés / 60 s
ágankénti becsült járműszám (W, E, S)	3	/ az ág tárolókapacitása (~7 m / jármű)
aktuális fázis (one-hot)	3	0 vagy 1
az aktuális zöld eddigi hossza	1	/ 120 s

Az elemek neveit az `env.observation_names()` adja vissza, pl. `'E_C_1_stopline.occupancy'`, `'approach_S.vehicles'`, `'phase_2'`.

Ha saját megfigyelést szeretnél (pl. csak a stopvonali hurkokat), írj egy `gymnasium.ObservationWrapper`-t, és használd az `info["detectors"]` nyers adatait:

```
import gymnasium as gym
import numpy as np

class OnlyStopline(gym.ObservationWrapper):
    def __init__(self, env):
        super().__init__(env)
        self.idx = [i for i, n in enumerate(env.unwrapped.observation_names())
                    if "stopline" in n or n.startswith("phase_")]
        self.observation_space = gym.spaces.Box(0, 1, (len(self.idx),), np.float32)

    def observation(self, obs):
        return obs[self.idx]
```

3.4. A nyers detektoradatok: `info["detectors"]`

Egy `DetectorSnapshot` objektum az utolsó lépés aggregált adataival:

```
snap = info["detectors"]
snap.duration                          # a lépés hossza [s]
snap.count, snap.occupancy, snap.speed, snap.gap   # numpy tömbök, detektoronként
snap.approach_vehicles                 # {'W': 12.0, 'E': 4.0, 'S': 7.0} - becslés a lépés végén
snap.approach_vehicle_seconds          # ugyanez a lépés másodperceire összegezve
snap.select(kind="stopline")           # bool maszk a szűréshez
snap.select(lane_id="E_C_1")
snap.by_id()["E_C_1_stopline"]         # {'count': 2.0, 'occupancy': 0.31, ...}
```

3.5. Amit szándékosan NEM kap meg az ágens

- jármű-azonosítók, pozíciók, úticélok,
- pontos sorhossz és várakozási idő,
- emisszió, utazási idő.

Ezeket a környezet `ground_truth=True` (alapértelmezett) beállítással az `info["ground_truth"]` alatt kiírja, de **csak kiértékeléshez** (pl. a dolgozatban annak bemutatására, hogy a tanult vezérlő tényleg csökkenti-e a késést). A megfigyelésbe vagy a jutalomba beépíteni „csalás”, mert a valóságban nem elérhető.

`info["ground_truth"]` kulcs	Jelentés
`halting_vehicle_seconds`	álló járművek száma másodpercenként összegezve (a bejövő ágakon)
`mean_delay_per_vehicle`	ugyanez elosztva az elindult járművek számával [s]
`mean_travel_time`	a célba ért járművek átlagos utazási ideje [s]
`co2_g`	CO2-kibocsátás a bejövő ágakon [g]
`departed, arrived`	elindult / célba ért járművek
`waiting_to_enter`	a hálózatba be sem tudó (visszatorlódott) járművek

4. Jutalomvektor: célfüggvények

A többcélú tanulásban a jutalom nem egy szám, hanem egy **vektor**: minden elem egy-egy célt mér. A célok általában ellentmondanak egymásnak (pl. kevés várakozás vs. kevés fázisváltás), ezért nincs egyetlen „legjobb” vezérlés, hanem **Pareto-optimális** vezérlések halmaza van.

A célokat a környezet létrehozásakor választod ki, a jutalomvektor ebben a sorrendben tartalmazza őket:

```
env = TJunctionMOEnv(objectives=("queue", "fairness", "switches"))   # alapértelmezés
env.objective_names    # ('queue', 'fairness', 'switches')
```

4.1. Beépített célfüggvények

Konvenció: **nagyobb = jobb** (a költségek negatív előjelűek). Minden érték a lépés másodperceire összegzett mennyiség, így az epizód összjutalma a teljes időre vett integrál, függetlenül attól, hogy a lépések milyen hosszúak. A `skála` csak arra szolgál, hogy a tipikus értékek 1 körüliek legyenek.

Név	Képlet (lépésenként)	Skála	Mit jelent
`queue`	$-\Sigma$_ágak Σ_másodpercek becsült járműszám	0,01	a teljes késleltetés közelítése (kevés jármű a csomópont előtt)
`fairness`	−max_ág Σ_másodpercek becsült járműszám	0,02	méltányosság: a legrosszabb helyzetű ág se várjon sokat
`switches`	−1, ha volt fázisváltás	1	stabil vezérlés, kevés elveszett idő az átmenetekben
`throughput`	a stopvonali hurkokon áthaladt járművek	0,1	áteresztőképesség (főleg túltelített helyzetben számít)
`stopline_occupancy`	$-\Sigma$ stopvonali foglalt másodpercek	0,1	a stopvonalnál álló járművek közelítése

Mindegyik csak hurokdetektor-adatokból (és a vezérlő saját állapotából) számol. A listát a programból is lekérheted:

```
from sumo_morl import list_objectives
print(list_objectives())
```

4.2. Saját célfüggvény

Egy célfüggvény egy függvény `(snap, ctx) -> float`, ahol

- `snap` a lépés hurokadatai (`DetectorSnapshot`, lásd Megfigyelés),
- `ctx` egy szótár: `phase_changed`, `phase`, `duration`, `time`, `forced`.

```
from sumo_morl import TJunctionMOEnv, register_objective

@register_objective("side_road_queue", scale=0.02,
                    doc="A déli mellékág becsült járműszáma (min.)")
def side_road_queue(snap, ctx):
    return -snap.approach_vehicle_seconds["S"]

env = TJunctionMOEnv(objectives=("queue", "side_road_queue"))
```

Fontos: a célfüggvényben **ne** kérdezd le a SUMO-t közvetlenül (TraCI), mert akkor valóságban nem mérhető adatot használnál.

4.3. Skalarizálás

A legtöbb RL algoritmus skalár jutalmat vár. A két legegyszerűbb módszer:

- **Lineáris skalarizálás** (`LinearScalarization` wrapper): `r = w · r_vec`, rögzített súlyvektorral. Különböző súlyokkal külön ágenseket tanítva feltérképezhető a Pareto-front (konvex része).
- **Súly-kondicionált ágens** (`WeightConditioned` wrapper): epizódonként véletlen súlyt sorsol, és azt a megfigyelés végére fűzi. Egyetlen háló tanulja meg az összes kompromisszumot; kiértékeléskor a kívánt súlyt kell megadni.

Mindkettő az eredeti vektort az `info["vector_reward"]` alatt is visszaadja.

Fejlettebb MORL módszerek (pl. Pareto Q-learning, Envelope Q-learning, nemlineáris hasznossági függvények) a vektor-jutalommal közvetlenül megvalósíthatók.

5. A környezet API-ja

```python
from sumo_morl import TJunctionMOEnv, Demand

env = TJunctionMOEnv(
    objectives=("queue", "fairness", "switches"),  # jutalomkomponensek
    demand=None,               # None = epizódonként véletlen igény
    episode_length=3600,       # szimulált másodperc
    decision_interval=5,       # "maradok" akció: ennyi másodperccel hosszabb zöld
    min_green=5,               # váltás után ennyi ideig biztosan zöld
    max_green=None,            # pl. 90: ennél hosszabb zöld után kényszerített váltás
    warmup=0,                  # ennyi s fix idejű bemelegítés az epizód elején
    gui=False,                 # sumo-gui megjelenítés
    seed=None,
    ground_truth=True,         # info["ground_truth"] kiértékeléshez
    record_signals=False,      # env.signal_log: másodpercenkénti jelzéskép (jelzésterv-ábrához)
)
```

A környezet a Gymnasium API-t és az MO-Gymnasium konvenciót követi (vektor jutalom + `env.reward_space`). `gym.make("sumo_morl/TJunction-v0", ...)` is működik.

5.1. Akció

`env.action_space = Discrete(3)`: a **következő fázis** sorszáma.

- Ha az akció az aktuális fázis → a zöld `decision_interval` (5) másodperccel meghosszabbodik.
- Ha más fázis → lefut a szabályos átmenet (4–6 s, lásd jelzésterv), majd az új fázis `min_green` (5) másodpercig biztosan zöld. Csak ezután jön a következő döntés.

Emiatt a lépések hossza **változó** (5 s vagy 9–11 s). Az `info["elapsed"]` megadja az adott lépés hosszát. A jutalmak a lépés másodperceire összegzettek, így ez nem torzítja az összjutalmat. (Ha diszkontálsz, érdemes lehet a diszkontot a lépés hosszához igazítani: `gamma ** info["elapsed"]` – ez egy jó haladó feladat.)

Bármelyik akció mindig érvényes; szabálytalan jelzéskép nem állhat elő.

5.2. Lépés és epizód

```python
obs, info = env.reset(seed=0)                          # új epizód, új forgalom
obs, reward, terminated, truncated, info = env.step(action)
```

- `reward`: `np.ndarray`, alakja `(len(objectives),)`.
- `terminated` mindig `False`; `truncated` akkor `True`, ha letelt az `episode_length`.
- Azonos `seed` → azonos forgalom (azonos járművek, azonos időpontokban), így a vezérlők igazságosan összehasonlíthatók.
- `env.close()` leállítja a SUMO-t. Több környezet futhat párhuzamosan.

5.3. Az `info` szótár

Kulcs	Tartalom
`time`	eltelt szimulációs idő az epizód kezdete óta [s]
`elapsed`	az utolsó lépés hossza [s]
`phase`, `phase_name`	az aktuális fázis
`time_in_phase`	az aktuális zöld eddigi hossza [s]
`phase_changed`, `forced_switch`	volt-e váltás / kényszerített volt-e (`max_green`)
`switches`	eddigi váltások száma az epizódban
`objectives`	a jutalomvektor névvel: `{'queue': -1.2, ...}`
`detectors`	a nyers hurokadatok (`DetectorSnapshot`)
`demand`	az epizód forgalmi igénye (`Demand`)
`ground_truth`	valós SUMO mutatók – **csak kiértékeléshez**

5.4. Forgalmi igény

```python
from sumo_morl import Demand

Demand.constant(main=600, side=250)                # állandó igény
Demand({("W", "E"): 500, ("W", "S"): 100, ("E", "W"): 450,
        ("E", "S"): 150, ("S", "W"): 120, ("S", "E"): 80})
Demand.constant(500, 200).scaled(1.3)              # 30%-kal több forgalom
Demand.random(rng, profile=(0.5, 1.0, 1.5, 1.0))   # csúcsórás hullám

env = TJunctionMOEnv(demand=Demand.constant(600, 250))
env = TJunctionMOEnv(demand=lambda rng: Demand.random(rng, main_range=(400, 800)))
env.reset(options={"demand": Demand.constant(300, 100)})   # egy epizódra felülírva
```

Mozgások: `("W","E")`, `("W","S")`, `("E","W")`, `("E","S")`, `("S","W")`, `("S","E")`; egység: jármű/óra. A járművek véletlenszerűen (Poisson-folyamat szerint) érkeznek.

Alapértelmezés (véletlen igény): főirány 200–700 jármű/óra irányonként, mellékág 80–400 jármű/óra, kanyarodási arányok 10–45%.

5.5. Megjelenítés

`TJunctionMOEnv(gui=True)` – elindul a `sumo-gui`, a szimuláció lassítva fut. Tanításhoz ne használd, mert lassú.

6. MORL segédfüggvények és összehasonlító vezérlők

Minden alább felsorolt függvény a `sumo_morl` csomagból importálható.

6.1. Epizód futtatása, kiértékelés

```python
from sumo_morl import run_episode, evaluate_policy

res = run_episode(env, policy, seed=0, record=True)
res.vector_return        # np.ndarray - az epizód vektor-hozama
res.ground_truth         # valós mutatók (kiértékeléshez)
res.history              # lépésenkénti napló (record=True esetén)

stats = evaluate_policy(env, policy, seeds=[0, 1, 2])
stats["mean"], stats["std"], stats["returns"], stats["ground_truth"]
```

A `policy` bármilyen függvény, amely `policy(obs, info) -> akció`. Egy Stable-Baselines3 modellből az `sb3_policy(model)` készít ilyet.

Az `evaluate_policy` azonos `seeds` listával minden vezérlőt **azonos forgalmon** futtat – összehasonlításnál mindig így tegyél.

6.2. Skalarizálás és súlyok

```
from sumo_morl import LinearScalarization, WeightConditioned, equally_spaced_weights,
    random_weights

env_s = LinearScalarization(env, weights=[0.5, 0.3, 0.2])     # skalár jutalom
env_w = WeightConditioned(env)                                 # véletlen súly epizódonként, a
    megfigyelésben

equally_spaced_weights(3, 4)    # egyenletes rács a súly-szimplexen (15 súlyvektor)
random_weights(3, 10)           # 10 véletlen súlyvektor (Dirichlet)
```

6.3. Pareto-front és hipertérfogat

```
from sumo_morl import pareto_mask, pareto_front, hypervolume, plot_pareto

points = np.array([...])          # pl. több vezérlő átlagos vektor-hozama (sorok)
pareto_mask(points)               # mely pontok nem domináltak (maximalizálás!)
pareto_front(points)
hypervolume(points, ref=points.min(axis=0) - 1)   # nagyobb = jobb front
plot_pareto(points, env.objective_names, labels=[...]).savefig("pareto.png")
```

- Egy `a` pont **dominálja** `b`-t, ha minden célban legalább olyan jó és legalább egyben jobb.
- A **hipertérfogat** a front és egy (mindennél rosszabb) referenciapont közötti térfogat – egyetlen számmal jellemzi, mennyire jó és mennyire szerteágazó a front. Különböző módszereket **azonos referenciaponttal** hasonlíts össze.

6.4. Ábrák

```
from sumo_morl.visualize import plot_scenario, plot_phases, plot_intergreen, plot_episode

res = run_episode(env, policy, seed=0, record=True)
plot_episode(res.history, env.phase_names).savefig("epizod.png")   # fázisok, sorok, jutalmak

# klasszikus jelzésterv-ábra (sávos diagram) - ehhez: TJunctionMOEnv(record_signals=True)
from sumo_morl.visualize import plot_signal_timeline
plot_signal_timeline(env.signal_log, env.plan, t_from=0, t_to=150).figure.savefig("jelzesterv.png")
```

6.5. Összehasonlító vezérlők (baseline-ok)

Mindegyik csak detektoradatot használ, és `policy(obs, info)` alakú.

Osztály	Működés
`RandomPolicy(env)`	véletlen fázis
`FixedTimePolicy(env, green_times=[30, 10, 15])`	fix idejű, ciklikus program
`ActuatedPolicy(env, max_green=45, gap=3)`	járműkövető: amíg az előjelző hurkon `gap` s-on belül jön jármű, hosszabbít; utána a következő igényelt fázisra vált
`LongestQueuePolicy(env, min_hold=10)`	mohó: a legnagyobb becsült igényű fázist választja

Egy jó MORL ágensnek legalább a fix idejű vezérlést kell vernie, és érdemes a járműkövetővel is összevetni. Egy tipikus eredmény (1800 s, 3 véletlen forgalom):

vezérlő	queue	fairness	switches	átl. késés [s]
véletlen	−463	−735	−148	59
fix 30/10/15	−175	−226	−78	17
járműkövető	−136	−157	−121	9
leghosszabb sor	−134	−156	−112	9

Látszik a kompromisszum: a fix idejű program kevesebbet vált, de több a várakozás.

6.6. Párhuzamos környezetek

Minden `TJunctionMOEnv` saját SUMO-folyamatot indít, így több is futhat egyszerre, pl. Stable-Baselines3 `SubprocVecEnv`-vel:

```
from stable_baselines3.common.vec_env import SubprocVecEnv
from sumo_morl import TJunctionMOEnv, LinearScalarization

def make(i):
    return lambda: LinearScalarization(TJunctionMOEnv(seed=i), [0.5, 0.3, 0.2])

vec_env = SubprocVecEnv([make(i) for i in range(4)])
```

6.7. Sebesség

Egy 3600 s-os epizód egy átlagos gépen nagyjából 6 másodperc alatt fut le (ground truth nélkül gyorsabb: `ground_truth=False`). Tanításhoz érdemes `ground_truth=False`-t használni, kiértékeléshez `True`-t.

7. Feladatötletek hallgatóknak

Nehézség szerint növekvő sorrendben.

7.1. Bevezető

1. **Ismerkedés.** Futtasd a `00_demo_fix_ido.py` demót és az `01_ismerkedes.py` példát `--gui` kapcsolóval. Figyeld meg az átmeneteket (sárga, piros-sárga). Melyik fázisváltás a leghosszabb, és miért?
2. **Baseline-ok.** Futtasd a `02_baseline_osszehasonlitas.py`-t. Keress olyan fix idejű zöldidő-kombinációt, amely a `queue` célban jobb a megadottaknál! Melyik másik cél romlik közben?

7.2. Egycélú → többcélú

3. **Lineáris skalarizálás.** Taníts DQN-t vagy PPO-t 5–10 különböző súlyvektorral (`03_sb3_sulyozott_tanitas.py`). Rajzold fel a Pareto-frontot, és számold ki a hipertérfogatot. Vesd össze a baseline-okkal.
4. **Súly-kondicionált ágens.** Taníts egyetlen ágenst `WeightConditioned` wrapperrel, majd értékeld ki ugyanazokkal a súlyokkal, mint a 3. feladatban. Melyik front jobb? Mennyivel kevesebb tanítás kellett?
5. **Saját cél.** Definiálj új célfüggvényt (pl. a mellékág várakozása, vagy a főirány haladási sebessége az előjelző hurkokon), és vizsgáld meg, hogyan változik a front.

7.3. Haladó

6. **Valódi MORL algoritmus.** Implementálj egy vektor-Q-értékes módszert (pl. Envelope Q-learning vagy Pareto-alapú DQN) a vektor-jutalomra.
7. **Nemlineáris hasznosság.** Optimalizálj a lineáris helyett pl. Csebisev (Tchebycheff) skalarizálással. Talál-e a front nem konvex részén pontot?
8. **Változó lépéshossz.** A lépések hossza változik (5 s vagy 9–11 s). Írj wrappert, amely a diszkontot `gamma ** info["elapsed"]` szerint alkalmazza (SMDP), és vizsgáld meg a hatását.
9. **Robusztusság.** Tanítsd az ágenst csak alacsony forgalmon, majd értékeld ki csúcsórás profillal (`Demand.random(rng, profile=(0.5, 1, 1.5, 1))`). Hogyan romlik a teljesítmény?
10. **Kevesebb szenzor.** Mi történik, ha egy detektortípust elhagysz a megfigyelésből (pl. csak stopvonali hurkok)? Melyik detektor a legfontosabb az ágensnek?

7.4. Kiértékelési tanácsok

- Mindig **azonos forgalmon** (azonos `seed`) hasonlíts össze vezérlőket.
- Több epizód (legalább 3–5) átlagát és szórását add meg.
- A dolgozatban a jutalmon kívül a valós mutatókat (`info["ground_truth"]`, pl. átlagos késés) is érdemes közölni, de ezeket ne használd tanításhoz.

A. A teljes forráskód

A.1. Csomag belépési pont: `sumo_morl/__init__.py`

```python
"""
sumo_morl - többcélú megerősítéses tanulás (MORL) jelzőlámpa-vezérléshez SUMO-ban.

Gyors kezdés::

    from sumo_morl import TJunctionMOEnv

    env = TJunctionMOEnv(objectives=("queue", "fairness", "switches"))
    obs, info = env.reset(seed=0)
    obs, reward_vec, terminated, truncated, info = env.step(env.action_space.sample())
    env.close()

Részletek: ``docs/sumo_morl/`` (magyar nyelvű dokumentáció).
"""
from .detectors import DetectorInfo, DetectorSnapshot
from .env import DEFAULT_OBJECTIVES, TJunctionMOEnv
from .intergreen import SignalPlan, build_signal_plan
from .morl import (
    EpisodeResult,
    LinearScalarization,
    WeightConditioned,
    equally_spaced_weights,
    evaluate_policy,
    hypervolume,
    pareto_front,
    pareto_mask,
    plot_pareto,
    random_weights,
    run_episode,
    sb3_policy,
)
from .objectives import OBJECTIVES, list_objectives, register_objective
from .policies import ActuatedPolicy, FixedTimePolicy, LongestQueuePolicy, Policy, RandomPolicy
from .scenario import TJunctionScenario
from .traffic import Demand

__version__ = "0.1.0"


def make_env(**kwargs):
    """Rövidítés: ``TJunctionMOEnv(**kwargs)``."""
    return TJunctionMOEnv(**kwargs)


try:  # regisztráció a gymnasium-ba: gym.make("sumo_morl/TJunction-v0")
    import gymnasium as _gym

    _gym.register(id="sumo_morl/TJunction-v0", entry_point="sumo_morl.env:TJunctionMOEnv",
                  disable_env_checker=True)  # a vektor-jutalom miatt
except Exception:  # pragma: no cover
    pass

__all__ = [
    "TJunctionMOEnv", "TJunctionScenario", "Demand", "SignalPlan", "build_signal_plan",
    "DetectorInfo", "DetectorSnapshot", "DEFAULT_OBJECTIVES", "OBJECTIVES", "list_objectives",
    "register_objective", "LinearScalarization", "WeightConditioned", "equally_spaced_weights",
    "random_weights", "pareto_mask", "pareto_front", "hypervolume", "plot_pareto", "run_episode",
    "evaluate_policy", "sb3_policy", "EpisodeResult", "Policy", "RandomPolicy", "FixedTimePolicy",
    "ActuatedPolicy", "LongestQueuePolicy", "make_env",
]
```

A.2. A környezet: sumo_morl/env.py

```python
"""
Többcélú (MORL) Gymnasium környezet a T-csomópont jelzőlámpájához.

A környezet az MO-Gymnasium konvenciót követi: a ``step`` jutalma egy
``numpy`` **vektor** (célfüggvényenként egy elem), és van ``reward_space``
attribútuma. Így közvetlenül használható pl. a ``morl-baselines`` könyvtár
algoritmusaival, illetve skalarizáló wrapperrel bármely egycélú RL
könyvtárral (pl. Stable-Baselines3).

Vezérlés (fázis alapú)
----------------------
* Akció: a következő fázis sorszáma (``Discrete(n_phases)``).
* Ha az akció = aktuális fázis -> a zöld ``decision_interval`` másodperccel
  meghosszabbodik.
* Ha más fázist választ -> lefut az átmenet (sárga / piros / piros-sárga), amit
  a közbenső idők határoznak meg, majd az új fázis ``min_green`` másodpercig
  biztosan zöld marad. Csak ezután jön a következő döntés.

Így az ágens **nem tud** szabálytalan jelzésképet előállítani, és nem kell
foglalkoznia a közbenső időkkel.

Megfigyelés (csak hurokdetektorok + a vezérlő saját állapota)
-------------------------------------------------------------
Lásd :meth:`TJunctionMOEnv.observation_names`.
"""
from __future__ import annotations

import contextlib
import io
import os
import shutil
import sys
import tempfile
import uuid
from typing import Any, Dict, List, Optional, Sequence, Tuple, Union

import gymnasium as gym
import numpy as np
from gymnasium import spaces

from .detectors import DetectorReader, DetectorSnapshot, FEATURES, write_detector_file
from .network import INCOMING_EDGES
from .objectives import get_objective
from .scenario import TJunctionScenario
from .sumo_utils import find_sumo_binary, sumo_home
from .traffic import Demand

DEFAULT_OBJECTIVES = ("queue", "fairness", "switches")


def _import_traci():
    home = sumo_home()
    if home and os.path.join(home, "tools") not in sys.path:
        sys.path.append(os.path.join(home, "tools"))
    import traci  # noqa: WPS433

    return traci


class TJunctionMOEnv(gym.Env):
    """Többcélú jelzőlámpa-vezérlési környezet egy T-csomópontra.

    Args:
```

```
        objectives: a célfüggvények nevei (lásd :func:`sumo_morl.list_objectives`).
            A jutalomvektor ebben a sorrendben tartalmazza őket.
        demand: forgalmi igény. ``None`` -> minden epizódban véletlen
            (:meth:`Demand.random`); megadható egy :class:`Demand` objektum vagy
            egy függvény ``rng -> Demand``.
        episode_length: az epizód hossza szimulált másodpercben.
        decision_interval: zöldhosszabbítás egy "maradok" akcióra [s].
        min_green: minimális zöldidő váltás után [s].
        max_green: ha nem ``None``, ennél hosszabb zöld után a környezet
            kikényszeríti a következő fázisra váltást [s].
        warmup: ennyi másodpercig fix idejű vezérlés fut az epizód elején, hogy
            a csomópont ne üresen induljon [s].
        gui: ``sumo-gui`` indítása (megjelenítés).
        seed: véletlenmag.
        ground_truth: ha igaz, az ``info["ground_truth"]`` alatt SUMO-ból
            kiolvasott "valós" mutatók is megjelennek (várakozási idő, CO2, ...).
            Ezeket **csak kiértékelésre** használd, a megfigyelésben és a
            jutalomban nem szerepelnek!
        scenario: egyedi forgatókönyv (haladóknak).
        sumo_args: további parancssori kapcsolók a SUMO-nak.
        record_signals: ha igaz, az ``env.signal_log`` másodpercenként rögzíti a
            jelzésképet (``[(idő, állapot), ...]``), pl. jelzésterv-ábrához.
    """

    metadata = {"render_modes": ["human"]}

    def __init__(
        self,
        objectives: Sequence[str] = DEFAULT_OBJECTIVES,
        demand: Union[None, Demand, Any] = None,
        episode_length: int = 3600,
        decision_interval: int = 5,
        min_green: int = 5,
        max_green: Optional[int] = None,
        warmup: int = 0,
        gui: bool = False,
        seed: Optional[int] = None,
        ground_truth: bool = True,
        scenario: Optional[TJunctionScenario] = None,
        sumo_args: Sequence[str] = (),
        render_mode: Optional[str] = None,
        record_signals: bool = False,
    ):
        super().__init__()
        self.scenario = scenario or TJunctionScenario()
        self.plan = self.scenario.plan
        self.objective_names = tuple(objectives)
        self._objectives = [get_objective(o) for o in self.objective_names]
        self.demand_spec = demand
        self.episode_length = int(episode_length)
        self.decision_interval = int(decision_interval)
        self.min_green = int(min_green)
        self.max_green = max_green
        self.warmup = int(warmup)
        self.gui = gui or render_mode == "human"
        self.render_mode = render_mode
        self.ground_truth = ground_truth
        self.sumo_args = list(sumo_args)
        self.record_signals = record_signals
        self.signal_log: List[Tuple[int, str]] = []

        self.detectors = self.scenario.detectors
        self.approaches = list(self.scenario.approaches)
        self._reader = DetectorReader(self.detectors, self.approaches)
        self._approach_capacity = self._compute_capacity()
```

```
129
130         n_ph = self.plan.n_phases
131         self.action_space = spaces.Discrete(n_ph)
132         self._obs_names = self._build_obs_names()
133         self.observation_space = spaces.Box(0.0, 1.0, shape=(len(self._obs_names),), dtype=np.
    float32)
134         self.reward_space = spaces.Box(-np.inf, np.inf, shape=(len(self._objectives),), dtype=np.
    float32)
135         self.reward_dim = len(self._objectives)
136
137         self._workdir = tempfile.mkdtemp(prefix="sumo_morl_")
138         self._det_file = write_detector_file(self.detectors, os.path.join(self._workdir, "det.add.
    xml"))
139         self._route_file = os.path.join(self._workdir, "routes.rou.xml")
140         self._label = f"sumo_morl_{uuid.uuid4().hex[:8]}"
141         self._traci = _import_traci()
142         self._conn = None
143         self._rng = np.random.default_rng(seed)
144         self._episode = 0
145         self.current_demand: Optional[Demand] = None
146
147     # ================================================================ tulajdonságok
148     @property
149     def n_phases(self) -> int:
150         return self.plan.n_phases
151
152     @property
153     def phase_names(self) -> List[str]:
154         return list(self.plan.phase_names)
155
156     def observation_names(self) -> List[str]:
157         """A megfigyelésvektor elemeinek nevei (ugyanabban a sorrendben)."""
158         return list(self._obs_names)
159
160     def _build_obs_names(self) -> List[str]:
161         names = []
162         for d in self.detectors:
163             for f in FEATURES:
164                 names.append(f"{d.id}.{f}")
165         for a in self.approaches:
166             names.append(f"approach_{a}.vehicles")
167         for p in range(self.plan.n_phases):
168             names.append(f"phase_{p}")
169         names.append("time_in_phase")
170         return names
171
172     def _compute_capacity(self) -> Dict[str, float]:
173         cap = {a: 0.0 for a in self.approaches}
174         lanes = {}
175         for d in self.detectors:
176             lanes.setdefault(d.lane_id, {})[d.kind] = d
177         for lane in lanes.values():
178             length = lane["stopline"].pos - lane["entry"].pos
179             cap[lane["entry"].approach] += length / 7.0  # ~7 m / jármű sorban
180         return cap
181
182     # ================================================================ SUMO
183     def _start_sumo(self, seed: int) -> None:
184         binary = find_sumo_binary("sumo-gui" if self.gui else "sumo")
185         cmd = [
186             binary, "-n", self.scenario.net_file, "-r", self._route_file, "-a", self._det_file,
187             "--step-length", "1", "--seed", str(seed), "--no-step-log", "true",
188             "--no-warnings", "true", "--time-to-teleport", "-1",
189             "--waiting-time-memory", "3600", "--duration-log.disable", "true",
190             "--end", str(self.episode_length + self.warmup + 3600),
```

```
        ]
        if self.gui:
            cmd += ["--start", "true", "--quit-on-end", "true", "--delay", "50"]
        cmd += self.sumo_args
        with contextlib.redirect_stdout(io.StringIO()):  # "Retrying..." üzenetek elnyelése
            self._traci.start(cmd, label=self._label)
        self._conn = self._traci.getConnection(self._label)
        self._reader.subscribe(self._conn)

    def _close_sumo(self) -> None:
        if self._conn is not None:
            try:
                self._conn.close()
            except Exception:
                pass
            self._conn = None

    def _make_demand(self) -> Demand:
        spec = self.demand_spec
        if spec is None:
            return Demand.random(self._rng)
        if isinstance(spec, Demand):
            return spec
        if callable(spec):
            return spec(self._rng)
        raise TypeError("demand: None, Demand vagy függvény (rng -> Demand) lehet.")

    # ================================================================ gym API
    def reset(self, *, seed: Optional[int] = None, options: Optional[Dict] = None):
        super().reset(seed=seed)
        if seed is not None:
            self._rng = np.random.default_rng(seed)
        options = options or {}
        self._close_sumo()
        self.current_demand = options.get("demand") or self._make_demand()
        sumo_seed = int(self._rng.integers(0, 2**31 - 1))
        self.current_demand.write_routes(self._route_file, self.episode_length + self.warmup,
    sumo_seed)
        self._start_sumo(sumo_seed)

        self._reader.reset()
        self.signal_log: List[Tuple[int, str]] = []
        self._phase = 0
        self._time_in_phase = 0
        self._sim_time = 0
        self._switches = 0
        self._gt_reset()
        self._set_state(self.plan.green_state(self._phase))

        # Bemelegítés fix idejű vezérléssel (a jutalomba nem számít bele).
        if self.warmup > 0:
            t0 = 0
            while t0 < self.warmup:
                target = (self._phase + 1) % self.n_phases if self._time_in_phase >= 20 else self.
    _phase
                if target != self._phase:
                    t0 += self._run_transition(target)
                else:
                    t0 += self._run_green(1)
            snap = self._reader.snapshot()
            self._gt_reset()
        else:
            snap = self._empty_snapshot()
        self._episode_time0 = self._sim_time
        self._last_snapshot = snap
```

```
        obs = self._observation(snap)
        info = self._info(snap, np.zeros(self.reward_dim, dtype=np.float32), False, 0)
        return obs, info

    def step(self, action: int):
        if self._conn is None:
            raise RuntimeError("Előbb hívd meg a reset()-et!")
        action = int(action)
        if not self.action_space.contains(action):
            raise ValueError(f"Érvénytelen akció: {action}")
        forced = (self.max_green is not None and action == self._phase
                  and self._time_in_phase >= self.max_green)
        if forced:
            action = (self._phase + 1) % self.n_phases

        changed = action != self._phase
        t_start = self._sim_time
        if changed:
            self._run_transition(action)
            self._run_green(self.min_green)
            self._switches += 1
        else:
            self._run_green(self.decision_interval)

        snap = self._reader.snapshot()
        self._last_snapshot = snap
        ctx = {"phase_changed": changed, "phase": self._phase, "duration": snap.duration,
               "time": self._sim_time, "forced": forced}
        reward = np.array([o(snap, ctx) for o in self._objectives], dtype=np.float32)
        truncated = (self._sim_time - self._episode_time0) >= self.episode_length
        obs = self._observation(snap)
        info = self._info(snap, reward, changed, self._sim_time - t_start, forced=forced)
        return obs, reward, False, truncated, info

    def close(self):
        self._close_sumo()
        if self._workdir and os.path.isdir(self._workdir):
            shutil.rmtree(self._workdir, ignore_errors=True)
            self._workdir = None

    def __del__(self):
        try:
            self.close()
        except Exception:
            pass

    def render(self):  # a sumo-gui maga rajzol
        return None

    # ============================================================== belső léptetés
    def _set_state(self, state: str) -> None:
        self._conn.trafficlight.setRedYellowGreenState(self.plan.tls_id, state)
        self._state = state

    def _sim_step(self) -> None:
        self._conn.simulationStep()
        self._sim_time += 1
        if self.record_signals:
            self.signal_log.append((self._sim_time, self._state))
        self._reader.sample(self._conn)
        if self.ground_truth:
            self._gt_sample()

    def _run_green(self, seconds: int) -> int:
        self._set_state(self.plan.green_state(self._phase))
```

```
        for _ in range(seconds):
            self._sim_step()
            self._time_in_phase += 1
        return seconds

    def _run_transition(self, target: int) -> int:
        total = 0
        for state, dur in self.plan.transition(self._phase, target):
            self._set_state(state)
            for _ in range(int(dur)):
                self._sim_step()
            total += int(dur)
        self._phase = target
        self._time_in_phase = 0
        return total

    # ============================================================== megfigyelés
    def _empty_snapshot(self) -> DetectorSnapshot:
        n = len(self.detectors)
        return DetectorSnapshot(0.0, self.detectors, np.zeros(n), np.zeros(n),
                                np.array([d.speed_limit for d in self.detectors]), np.zeros(n),
                                {a: 0.0 for a in self.approaches}, {a: 0.0 for a in self.approaches},
                                np.zeros(n))

    def _observation(self, snap: DetectorSnapshot) -> np.ndarray:
        dur = max(snap.duration, 1.0)
        feats = []
        for i, d in enumerate(self.detectors):
            flow = snap.count[i] / dur * 3600.0 / 1800.0       # ~ telítési forgalomhoz viszonyítva
            feats += [min(flow, 1.0), snap.occupancy[i], min(snap.speed[i] / d.speed_limit, 1.0),
                      min(snap.gap[i] / 60.0, 1.0)]
        for a in self.approaches:
            feats.append(min(snap.approach_vehicles[a] / max(self._approach_capacity[a], 1.0), 1.0))
        onehot = [0.0] * self.n_phases
        onehot[self._phase] = 1.0
        feats += onehot
        feats.append(min(self._time_in_phase / 120.0, 1.0))
        return np.clip(np.asarray(feats, dtype=np.float32), 0.0, 1.0)

    def _info(self, snap, reward, changed, elapsed, forced=False) -> Dict[str, Any]:
        info = {
            "time": self._sim_time - getattr(self, "_episode_time0", 0),
            "elapsed": elapsed,
            "phase": self._phase,
            "phase_name": self.phase_names[self._phase] if self.phase_names else str(self._phase),
            "time_in_phase": self._time_in_phase,
            "phase_changed": changed,
            "forced_switch": forced,
            "switches": self._switches,
            "objectives": {n: float(r) for n, r in zip(self.objective_names, reward)},
            "detectors": snap,
            "demand": self.current_demand,
        }
        if self.ground_truth:
            info["ground_truth"] = self._gt_summary()
        return info

    # ============================================================== "valós" mutatók
    def _gt_reset(self) -> None:
        self._gt = {"halting_vehicle_seconds": 0.0, "co2_g": 0.0, "arrived": 0, "departed": 0,
                    "travel_time_sum": 0.0}
        self._depart_time: Dict[str, int] = {}

```

```
    def _gt_sample(self) -> None:
        c = self._conn
        halting = 0
        co2 = 0.0
        for e in list(INCOMING_EDGES.values()):
            halting += c.edge.getLastStepHaltingNumber(e)
            co2 += c.edge.getCO2Emission(e)
        self._gt["halting_vehicle_seconds"] += halting
        self._gt["co2_g"] += co2 / 1000.0
        for v in c.simulation.getDepartedIDList():
            self._depart_time[v] = self._sim_time
        self._gt["departed"] += c.simulation.getDepartedNumber()
        for v in c.simulation.getArrivedIDList():
            t0 = self._depart_time.pop(v, None)
            if t0 is not None:
                self._gt["travel_time_sum"] += self._sim_time - t0
                self._gt["arrived"] += 1
        self._gt["waiting_to_enter"] = len(c.simulation.getPendingVehicles())

    def _gt_summary(self) -> Dict[str, float]:
        g = dict(self._gt)
        g["mean_travel_time"] = g["travel_time_sum"] / g["arrived"] if g["arrived"] else float("nan
    ")
        g["mean_delay_per_vehicle"] = (g["halting_vehicle_seconds"] / g["departed"]
                                       if g["departed"] else float("nan"))
        return g
```

A.3. Célfüggvények: `sumo_morl/objectives.py`

```
"""
Célfüggvények (a vektor-jutalom komponensei).

Minden célfüggvény egy függvény, amely egy vezérlési lépés hurokdetektor-adataiból
(:class:`~sumo_morl.detectors.DetectorSnapshot`) és néhány, a vezérlő által
amúgy is ismert adatból (pl. volt-e fázisváltás) **egyetlen számot** ad vissza.
A konvenció: **nagyobb = jobb** (a költségeket negatív előjellel adjuk vissza).

A jutalmak a lépés másodperceire összegzett mennyiségek, így a lépés hosszától
függetlenül összeadhatók (az epizód összjutalma = időintegrál). A ``scale``
tényező csak arra szolgál, hogy a tipikus értékek ~1 nagyságrendűek legyenek.

Saját célfüggvény hozzáadása::

    from sumo_morl import register_objective

    @register_objective("stopline_occupancy", scale=0.1,
                        doc="A stopvonali hurkok foglaltsága (álló járművek)")
    def stopline_occupancy(snap, ctx):
        mask = snap.select(kind="stopline")
        return -float(snap.stopline_occupied_seconds[mask].sum())

    env = TJunctionMOEnv(objectives=("queue", "stopline_occupancy"))
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Callable, Dict

from .detectors import DetectorSnapshot

ObjectiveFn = Callable[[DetectorSnapshot, Dict], float]


```

```python
@dataclass
class Objective:
    name: str
    fn: ObjectiveFn
    scale: float
    doc: str

    def __call__(self, snap: DetectorSnapshot, ctx: Dict) -> float:
        return float(self.fn(snap, ctx)) * self.scale


OBJECTIVES: Dict[str, Objective] = {}


def register_objective(name: str, scale: float = 1.0, doc: str = ""):
    """Dekorátor új célfüggvény regisztrálásához (lásd a modul leírását)."""

    def deco(fn: ObjectiveFn) -> ObjectiveFn:
        OBJECTIVES[name] = Objective(name, fn, scale, doc or (fn.__doc__ or "").strip())
        return fn

    return deco


def get_objective(name: str) -> Objective:
    if name not in OBJECTIVES:
        raise KeyError(f"Ismeretlen célfüggvény: '{name}'. Elérhetők: {sorted(OBJECTIVES)}")
    return OBJECTIVES[name]


def list_objectives() -> str:
    """Az elérhető célfüggvények listája rövid leírással."""
    return "\n".join(f"  {o.name:<12} (skála {o.scale:g}): {o.doc}" for o in OBJECTIVES.values())


# ------------------------------------------------------------------ beépítettek
@register_objective("queue", scale=0.01,
                    doc="-(becsült járműszám az ágakon, másodpercekre összegezve) "
                        "[jármű*s]; a teljes késleltetés közelítése (min.)")
def _queue(snap: DetectorSnapshot, ctx: Dict) -> float:
    return -sum(snap.approach_vehicle_seconds.values())


@register_objective("throughput", scale=0.1,
                    doc="A stopvonali hurkokon áthaladt járművek száma [jármű] (max.)")
def _throughput(snap: DetectorSnapshot, ctx: Dict) -> float:
    return float(snap.count[snap.select(kind="stopline")].sum())


@register_objective("fairness", scale=0.02,
                    doc="-(a legrosszabb ág becsült járműszáma, másodpercekre összegezve) "
                        "[jármű*s]; méltányosság az ágak között (min.)")
def _fairness(snap: DetectorSnapshot, ctx: Dict) -> float:
    return -max(snap.approach_vehicle_seconds.values()) if snap.approach_vehicle_seconds else 0.0


@register_objective("switches", scale=1.0,
                    doc="-1 minden fázisváltásnál; stabil, kevés váltásos vezérlés (min.)")
def _switches(snap: DetectorSnapshot, ctx: Dict) -> float:
    return -1.0 if ctx.get("phase_changed") else 0.0


@register_objective("stopline_occupancy", scale=0.1,
                    doc="-(a stopvonali hurkok foglalt másodpercei) [s]; a stopvonalnál "
                        "álló/lassú járművek közelítése (min.)")
```

```python
def _stopline_occ(snap: DetectorSnapshot, ctx: Dict) -> float:
    return -float(snap.stopline_occupied_seconds[snap.select(kind="stopline")].sum())
```

A.4. Hurokdetektorok: `sumo_morl/detectors.py`

```python
"""
Indukcióshurok-detektorok (E1) elhelyezése és olvasása.

**Csak valóságban is mérhető adatokat használunk.** Egy valódi hurokdetektor
azt érzékeli, hogy van-e jármű a hurok felett. Ebből a forgalomtechnikai
vezérlők a következőket számolják, és mi is pontosan ezeket adjuk át:

* ``count``      - a hurkon áthaladt járművek száma (felfutó élek),
* ``occupancy``  - foglaltság: az idő hányadában volt jármű a hurok felett [0..1],
* ``speed``      - átlagsebesség a hurkon [m/s] (dupla hurokkal mérhető;
  ha nem volt jármű, a megengedett sebességet adjuk vissza),
* ``gap``        - a legutóbbi érzékelés óta eltelt idő [s] (időrés).

Nem használunk SUMO "belső igazságot" (pl. jármű-azonosítók, várakozási idő,
pontos sorhossz). A jármű-azonosítókat a SUMO-ból csak arra használjuk, hogy a
hurok felfutó éleit megszámoljuk - ez egyenértékű egy valódi hurok
impulzusszámlálásával.

Minden bejövő sávon három hurok van::

    ág eleje                                                  stopvonal
    |--[entry]-----------------------------[advance]--------[stopline]|
       10 m-re az ág elejétől                40 m-re          3 m-re a stopvonaltól

Az ``entry`` és ``stopline`` hurkok számlálóiból ágankénti "bemenet-kimenet"
becslést készítünk a két hurok közötti járműszámra (ez a klasszikus,
valóságban is használt sorhossz-becslési módszer).
"""
from __future__ import annotations

import xml.etree.ElementTree as ET
from dataclasses import dataclass, field
from typing import Dict, List, Optional

import numpy as np

DETECTOR_KINDS = ("entry", "advance", "stopline")
FEATURES = ("count", "occupancy", "speed", "gap")


@dataclass
class DetectorInfo:
    id: str
    lane_id: str
    edge_id: str
    approach: str      # 'W', 'E', 'S'
    kind: str          # 'entry', 'advance', 'stopline'
    pos: float         # pozíció a sáv elejétől [m]
    speed_limit: float


def detector_layout(net_file: str, incoming_edges: Dict[str, str],
                    stopline_dist: float = 3.0, advance_dist: float = 40.0,
                    entry_dist: float = 10.0) -> List[DetectorInfo]:
    """Kiszámolja a hurkok helyét a bejövő sávokon."""
    root = ET.parse(net_file).getroot()
    edge_to_app = {e: a for a, e in incoming_edges.items()}
    dets: List[DetectorInfo] = []
```

```
    for edge in root.findall("edge"):
        eid = edge.get("id")
        if eid not in edge_to_app:
            continue
        for lane in edge.findall("lane"):
            lid, length, v = lane.get("id"), float(lane.get("length")), float(lane.get("speed"))
            for kind, pos in (("entry", entry_dist),
                              ("advance", length - advance_dist),
                              ("stopline", length - stopline_dist)):
                dets.append(DetectorInfo(f"{lid}_{kind}", lid, eid, edge_to_app[eid], kind, pos, v)
    )
    dets.sort(key=lambda d: ("WES".index(d.approach) if d.approach in "WES" else 9,
                             d.lane_id, DETECTOR_KINDS.index(d.kind)))
    return dets


def write_detector_file(dets: List[DetectorInfo], out_file: str) -> str:
    """SUMO additional fájl (``.add.xml``) írása a hurkokkal."""
    root = ET.Element("additional")
    for d in dets:
        ET.SubElement(root, "inductionLoop", id=d.id, lane=d.lane_id, pos=f"{d.pos:.2f}",
                      period="3600", file="NUL")
    ET.indent(root)
    ET.ElementTree(root).write(out_file, encoding="utf-8", xml_declaration=True)
    return out_file


@dataclass(eq=False)
class DetectorSnapshot:
    """Egy vezérlési lépés (több szimulációs másodperc) aggregált hurokadatai.

    Minden tömb a ``detectors`` lista sorrendjét követi.
    """

    duration: float                         # a lépés hossza [s]
    detectors: List[DetectorInfo]
    count: np.ndarray                       # áthaladt járművek [db]
    occupancy: np.ndarray                   # átlagos foglaltság [0..1]
    speed: np.ndarray                       # átlagsebesség [m/s]
    gap: np.ndarray                         # utolsó érzékelés óta [s] (lépés végén)
    approach_vehicles: Dict[str, float]   # bemenet-kimenet becslés ágankként [db] (lépés végén)
    approach_vehicle_seconds: Dict[str, float]  # a becslés időintegrálja a lépés alatt [db*s]
    stopline_occupied_seconds: np.ndarray = field(default=None)  # foglalt másodpercek [s]

    def __eq__(self, other) -> bool:
        if not isinstance(other, DetectorSnapshot):
            return NotImplemented
        arrs = ("count", "occupancy", "speed", "gap", "stopline_occupied_seconds")
        return (self.duration == other.duration and self.approach_vehicles == other.
    approach_vehicles
                and all(np.array_equal(getattr(self, a), getattr(other, a)) for a in arrs))

    def by_id(self) -> Dict[str, Dict[str, float]]:
        """``{detektor_id: {count, occupancy, speed, gap}}`` szótár."""
        return {d.id: {"count": float(self.count[i]), "occupancy": float(self.occupancy[i]),
                       "speed": float(self.speed[i]), "gap": float(self.gap[i])}
                for i, d in enumerate(self.detectors)}

    def select(self, kind: Optional[str] = None, lane_id: Optional[str] = None,
               approach: Optional[str] = None) -> np.ndarray:
        """Indexmaszk a detektorok szűréséhez (típus / sáv / ág szerint)."""
        return np.array([(kind is None or d.kind == kind) and
                         (lane_id is None or d.lane_id == lane_id) and
                         (approach is None or d.approach == approach)
                         for d in self.detectors], dtype=bool)
```

```python


class DetectorReader:
    """A hurkok másodpercenkénti olvasása TraCI-n keresztül és aggregálása."""

    def __init__(self, dets: List[DetectorInfo], approaches: List[str]):
        self.dets = dets
        self.approaches = list(approaches)
        self._idx_entry = {a: [i for i, d in enumerate(dets) if d.approach == a and d.kind == "entry"]
                           for a in self.approaches}
        self._idx_stop = {a: [i for i, d in enumerate(dets) if d.approach == a and d.kind == "stopline"]
                          for a in self.approaches}
        self.reset()

    def reset(self) -> None:
        n = len(self.dets)
        self._prev_ids: List[set] = [set() for _ in range(n)]
        self._gap = np.zeros(n)
        self.approach_vehicles = {a: 0.0 for a in self.approaches}
        self._begin_step()

    def _begin_step(self) -> None:
        n = len(self.dets)
        self._count = np.zeros(n)
        self._occ_sum = np.zeros(n)
        self._speed_sum = np.zeros(n)
        self._speed_n = np.zeros(n)
        self._seconds = 0
        self._veh_seconds = {a: 0.0 for a in self.approaches}

    def subscribe(self, conn) -> None:
        """TraCI előfizetés a hurkokra (gyorsabb, mint az egyenkénti lekérdezés)."""
        import traci.constants as tc

        self._vars = (tc.LAST_STEP_VEHICLE_ID_LIST, tc.LAST_STEP_OCCUPANCY,
                      tc.LAST_STEP_MEAN_SPEED, tc.LAST_STEP_TIME_SINCE_DETECTION)
        for d in self.dets:
            conn.inductionloop.subscribe(d.id, self._vars)

    def sample(self, conn) -> None:
        """Egy szimulációs lépés (1 s) utáni leolvasás. ``conn`` egy TraCI kapcsolat."""
        res = conn.inductionloop.getAllSubscriptionResults()
        v_ids, v_occ, v_spd, v_gap = self._vars
        new = np.zeros(len(self.dets))
        for i, d in enumerate(self.dets):
            r = res[d.id]
            ids = set(r[v_ids])
            new[i] = len(ids - self._prev_ids[i])  # felfutó élek = új járművek a hurkon
            self._prev_ids[i] = ids
            self._occ_sum[i] += r[v_occ] / 100.0
            sp = r[v_spd]
            if sp >= 0:
                self._speed_sum[i] += sp
                self._speed_n[i] += 1
            self._gap[i] = r[v_gap]
        self._count += new
        # bemenet-kimenet becslés ágankként: + entry hurok, - stopvonali hurok
        for a in self.approaches:
            v = (self.approach_vehicles[a] + new[self._idx_entry[a]].sum()
                 - new[self._idx_stop[a]].sum())
            self.approach_vehicles[a] = max(0.0, float(v))
            self._veh_seconds[a] += self.approach_vehicles[a]
        self._seconds += 1
```

```
185
186     def snapshot(self) -> DetectorSnapshot:
187         """Lezárja az aktuális lépést és visszaadja az aggregált adatokat."""
188         secs = max(1, self._seconds)
189         speed = np.array([self._speed_sum[i] / self._speed_n[i] if self._speed_n[i] > 0 else d.
    speed_limit
190                           for i, d in enumerate(self.dets)])
191         snap = DetectorSnapshot(
192             duration=float(self._seconds), detectors=self.dets,
193             count=self._count.copy(), occupancy=self._occ_sum / secs, speed=speed,
194             gap=self._gap.copy(), approach_vehicles=dict(self.approach_vehicles),
195             approach_vehicle_seconds=dict(self._veh_seconds),
196             stopline_occupied_seconds=self._occ_sum.copy(),
197         )
198         self._begin_step()
199         return snap
```

A.5. Jelzésterv a közbenső időkből: `sumo_morl/intergreen.py`

```
1  """
2  Jelzésterv (fázisok + közbenső idők) automatikus előállítása.
3
4  A közbenső időket (intergreen) **a repó meglévő közbensőidő-számító kódja**
5  (:mod:`geometry_utils` + :mod:`sumo_parser`) számolja ki a csomópont
6  geometriájából (ütközési pontok, kanyarodási sugarak, sebességek). Ez a modul
7  erre építve:
8
9  1. jelzőcsoportokat képez (egy bejövő sáv = egy jelzőcsoport, így egy sávon
10    belül soha nem kap egyik irány zöldet a másik nélkül),
11 2. a mozgás-szintű közbenső idő mátrixból jelzőcsoport-szintű mátrixot képez
12    (maximum a csoportok mozgásai között),
13 3. a konfliktus-mentes csoportokból (gráf klikkek) fázisokat képez,
14 4. bármely két fázis között legenerálja az átmenetet (sárga, piros,
15    piros-sárga), úgy hogy minden közbenső idő teljesüljön.
16
17 A hallgatóknak mindezzel nem kell foglalkozniuk: a környezet egy kész
18 :class:`SignalPlan` objektumot használ, az ágens csak a következő fázis
19 sorszámát választja ki.
20 """
21 from __future__ import annotations
22
23 import contextlib
24 import io
25 import os
26 import sys
27 import xml.etree.ElementTree as ET
28 from dataclasses import dataclass, field
29 from typing import Dict, List, Optional, Sequence, Tuple
30
31 import numpy as np
32
33 # A repó gyökerében lévő közbensőidő-számító modulok importálása.
34 _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
35 if _REPO_ROOT not in sys.path:  # pragma: no cover - csak forrásból futtatva kell
36     sys.path.insert(0, _REPO_ROOT)
37
38 DIR_NAMES = {"s": "egyenes", "l": "balra", "r": "jobbra", "t": "visszafordulás",
39              "L": "balra (enyhén)", "R": "jobbra (enyhén)"}
40
41
42 @dataclass
43 class Link:
44     """Egy SUMO "link": egy bejövő sávból egy kimenő sávba vezető mozgás."""
```

```
45
46      index: int          # a jelzőlámpa állapot-sztringjében elfoglalt pozíció
47      from_edge: str
48      from_lane: int
49      to_edge: str
50      direction: str      # 's', 'l', 'r', ...
51
52      @property
53      def lane_id(self) -> str:
54          return f"{self.from_edge}_{self.from_lane}"
55
56
57  @dataclass
58  class SignalGroup:
59      """Jelzőcsoport: egy bejövő sáv összes mozgása, amelyek együtt kapnak jelzést."""
60
61      name: str
62      lane_id: str
63      links: List[int]
64      description: str = ""
65
66
67  @dataclass
68  class SignalPlan:
69      """A csomópont teljes jelzésterve.
70
71      Attributes:
72          groups: jelzőcsoportok listája.
73          phases: fázisok; mindegyik a benne zöldet kapó csoportok indexeinek listája.
74          intergreen: ``[kiürítő][belépő]`` közbenső idő mátrix a csoportok között
75              [s]; ``nan``, ha a két csoport nem konfliktusos.
76          link_intergreen: a mozgás (link) szintű mátrix, ahogy a közbensőidő-számító
77              kód kiadta.
78          n_links: a SUMO állapot-sztring hossza.
79          yellow: sárga idő [s].
80          red_yellow: piros-sárga idő [s].
81      """
82
83      tls_id: str
84      links: List[Link]
85      groups: List[SignalGroup]
86      phases: List[List[int]]
87      intergreen: np.ndarray
88      link_intergreen: np.ndarray
89      yellow: int = 3
90      red_yellow: int = 2
91      phase_names: List[str] = field(default_factory=list)
92
93      @property
94      def n_links(self) -> int:
95          return len(self.links)
96
97      @property
98      def n_phases(self) -> int:
99          return len(self.phases)
100
101     def conflicts(self, gi: int, gj: int) -> bool:
102         """Igaz, ha a két jelzőcsoport konfliktusos (nem kaphat egyszerre zöldet)."""
103         return not np.isnan(self.intergreen[gi, gj]) or not np.isnan(self.intergreen[gj, gi])
104
105     # ------------------------------------------------------------------ állapotok
106     def _state(self, chars: Dict[int, str]) -> str:
107         s = ["r"] * self.n_links
108         for gi, ch in chars.items():
109             for li in self.groups[gi].links:
```

```
                s[li] = ch
        return "".join(s)

    def green_state(self, phase: int) -> str:
        """A ``phase`` fázis SUMO állapot-sztringje (pl. ``'GrrrGG'``)."""
        return self._state({g: "G" for g in self.phases[phase]})

    def phase_lanes(self, phase: int) -> List[str]:
        """A fázisban zöldet kapó bejövő sávok azonosítói."""
        return [self.groups[g].lane_id for g in self.phases[phase]]

    def group_start_offsets(self, from_phase: int, to_phase: int) -> Dict[int, int]:
        """Az induló csoportok zöldkezdete a régi zöld vége után [s]."""
        cur, nxt = set(self.phases[from_phase]), set(self.phases[to_phase])
        stopping, starting = cur - nxt, nxt - cur
        out = {}
        for gj in starting:
            need = self.red_yellow
            for gi in stopping:
                k = self.intergreen[gi, gj]
                if not np.isnan(k):
                    need = max(need, int(k))
            out[gj] = need
        return out

    def transition(self, from_phase: int, to_phase: int) -> List[Tuple[str, int]]:
        """Átmenet két fázis között: ``[(állapot, időtartam_s), ...]``.

        ``t = 0`` a régi zöld vége. A megszűnő csoportok ``yellow`` ideig sárgát,
        utána pirosat kapnak; az induló csoportok annyi idő után kapnak zöldet,
        amennyit a közbenső idő mátrix előír (legalább ``red_yellow``), előtte
        ``red_yellow`` ideig piros-sárgát. A megmaradó csoportok végig zöldek.
        Az átmenet akkor ér véget, amikor az utolsó induló csoport is zöldet kap.
        """
        if from_phase == to_phase:
            return []
        cur, nxt = set(self.phases[from_phase]), set(self.phases[to_phase])
        stopping, staying = cur - nxt, cur & nxt
        starts = self.group_start_offsets(from_phase, to_phase)
        total = max([self.yellow] + list(starts.values()))
        cuts = {0, total, min(self.yellow, total)}
        for t in starts.values():
            cuts.update({t - self.red_yellow, t})
        cuts = sorted(c for c in cuts if 0 <= c <= total)
        steps: List[Tuple[str, int]] = []
        for a, b in zip(cuts[:-1], cuts[1:]):
            if b <= a:
                continue
            chars: Dict[int, str] = {g: "G" for g in staying}
            for g in stopping:
                chars[g] = "y" if a < self.yellow else "r"
            for g, t in starts.items():
                if a >= t:
                    chars[g] = "G"
                elif a >= t - self.red_yellow:
                    chars[g] = "u"
                else:
                    chars[g] = "r"
            st = self._state(chars)
            if steps and steps[-1][0] == st:
                steps[-1] = (st, steps[-1][1] + (b - a))
            else:
                steps.append((st, b - a))
        # Az utolsó szakasz már maga az új fázis zöldje -> nem része az átmenetnek.
        if steps and steps[-1][0] == self.green_state(to_phase):
```

```
            steps.pop()
        return steps

    def transition_duration(self, from_phase: int, to_phase: int) -> int:
        return int(sum(d for _, d in self.transition(from_phase, to_phase)))

    # ------------------------------------------------------------------ leírás
    def describe(self) -> str:
        """Ember által olvasható összefoglaló a jelzéstervről."""
        lines = [f"Jelzőlámpa: {self.tls_id}", "", "Jelzőcsoportok:"]
        for i, g in enumerate(self.groups):
            lines.append(f"  [{i}] {g.name:<12} sáv={g.lane_id:<6} linkek={g.links}  {g.description}")
        lines.append("")
        lines.append("Közbenső idők [s] (sor = kiürítő, oszlop = belépő, '-' = nincs konfliktus):")
        hdr = "      " + "".join(f"{i:>5}" for i in range(len(self.groups)))
        lines.append(hdr)
        for i in range(len(self.groups)):
            row = "".join(
                f"{'-' if np.isnan(v) else int(v):>5}" for v in self.intergreen[i]
            )
            lines.append(f"  [{i}]{row}")
        lines.append("")
        lines.append("Fázisok:")
        for p, groups in enumerate(self.phases):
            name = self.phase_names[p] if p < len(self.phase_names) else ""
            lines.append(f"  Fázis {p}: {self.green_state(p)}  csoportok={groups}  {name}")
        lines.append("")
        lines.append("Átmenetek hossza [s]:")
        for a in range(self.n_phases):
            for b in range(self.n_phases):
                if a != b:
                    steps = " -> ".join(f"{s}({d}s)" for s, d in self.transition(a, b))
                    lines.append(f"  {a} -> {b}: {self.transition_duration(a, b):>2} s   {steps}")
        return "\n".join(lines)


# ============================================================================ építés
def read_links(net_file: str, tls_id: str) -> List[Link]:
    """Kiolvassa a jelzőlámpához tartozó linkeket a ``.net.xml``-ből."""
    root = ET.parse(net_file).getroot()
    links: Dict[int, Link] = {}
    for c in root.findall("connection"):
        if c.get("tl") == tls_id and c.get("linkIndex") is not None:
            idx = int(c.get("linkIndex"))
            links[idx] = Link(idx, c.get("from"), int(c.get("fromLane")), c.get("to"), c.get("dir"))
    return [links[i] for i in sorted(links)]


def compute_link_intergreen(net_file: str, tls_id: str) -> np.ndarray:
    """A meglévő közbensőidő-számító kóddal kiszámolja a link-szintű mátrixot.

    Returns:
        ``n_links x n_links`` mátrix, ``[kiürítő][belépő]`` közbenső idő [s],
        ``nan`` ahol nincs ütközési pont.
    """
    from geometry_utils import calculate_junction_data  # repó gyökér
    from sumo_parser import SumoInternalParser          # repó gyökér

    with contextlib.redirect_stdout(io.StringIO()):
        parser = SumoInternalParser(net_file)
    jdata = next(j for j in parser.junctions if j["id"] == tls_id)
    data = calculate_junction_data(jdata)
    n_links = len(read_links(net_file, tls_id))
```

```
    mat = np.full((n_links, n_links), np.nan)
    for i, li in enumerate(data["lines"]):
        for j, lj in enumerate(data["lines"]):
            v = data["matrix"][i][j]
            if v is None:
                continue
            a, b = li["logic_idx"], lj["logic_idx"]
            if 0 <= a < n_links and 0 <= b < n_links:
                mat[a, b] = np.nanmax([mat[a, b], float(v[0])])
    # Ha csak az egyik irányban talált ütközési pontot, a másik irányt is
    # konfliktusnak tekintjük (a legnagyobb ismert értékkel).
    for a in range(n_links):
        for b in range(n_links):
            if np.isnan(mat[a, b]) and not np.isnan(mat[b, a]):
                mat[a, b] = mat[b, a]
    return mat


def build_signal_groups(links: Sequence[Link], names: Optional[Dict[str, str]] = None) -> List[
    SignalGroup]:
    """Jelzőcsoportok képzése: egy bejövő sáv = egy csoport."""
    by_lane: Dict[str, List[Link]] = {}
    for l in links:
        by_lane.setdefault(l.lane_id, []).append(l)
    groups = []
    for lane_id in sorted(by_lane):
        ls = by_lane[lane_id]
        desc = ", ".join(f"{DIR_NAMES.get(l.direction, l.direction)} ({l.to_edge})" for l in ls)
        name = (names or {}).get(lane_id, lane_id)
        groups.append(SignalGroup(name, lane_id, [l.index for l in ls], desc))
    return groups


def group_intergreen(link_mat: np.ndarray, groups: Sequence[SignalGroup]) -> np.ndarray:
    """Csoport-szintű közbenső idő mátrix (maximum a csoportok linkjei között)."""
    n = len(groups)
    out = np.full((n, n), np.nan)
    for i, gi in enumerate(groups):
        for j, gj in enumerate(groups):
            if i == j:
                continue
            vals = [link_mat[a, b] for a in gi.links for b in gj.links if not np.isnan(link_mat[a,
    b])]
            if vals:
                out[i, j] = max(vals)
    return out


def phases_from_conflicts(intergreen: np.ndarray) -> List[List[int]]:
    """Maximális konfliktusmentes csoporthalmazok (klikkek) = fázisok."""
    import networkx as nx

    n = intergreen.shape[0]
    g = nx.Graph()
    g.add_nodes_from(range(n))
    for i in range(n):
        for j in range(i + 1, n):
            if np.isnan(intergreen[i, j]) and np.isnan(intergreen[j, i]):
                g.add_edge(i, j)
    cliques = [sorted(c) for c in nx.find_cliques(g)]
    cliques.sort(key=lambda c: (-len(c), c))
    return cliques


def build_signal_plan(
```

```
    net_file: str,
    tls_id: str,
    phases: Optional[List[List[int]]] = None,
    yellow: int = 3,
    red_yellow: int = 2,
    group_names: Optional[Dict[str, str]] = None,
    phase_names: Optional[List[str]] = None,
) -> SignalPlan:
    """Teljes jelzésterv előállítása egy ``.net.xml`` fájlból.

    Args:
        net_file: hálózat fájl.
        tls_id: a jelzőlámpás csomópont azonosítója.
        phases: opcionális kézi fázislista (csoport-indexek); ha ``None``, a
            konfliktusmentes klikkek lesznek a fázisok.
        yellow, red_yellow: sárga és piros-sárga idő [s].
    """
    links = read_links(net_file, tls_id)
    link_mat = compute_link_intergreen(net_file, tls_id)
    groups = build_signal_groups(links, group_names)
    ig = group_intergreen(link_mat, groups)
    if phases is None:
        phases = phases_from_conflicts(ig)
    plan = SignalPlan(tls_id, links, groups, phases, ig, link_mat, yellow, red_yellow,
                      list(phase_names or []))
    for p in phases:
        for a in p:
            for b in p:
                if a != b and plan.conflicts(a, b):
                    raise ValueError(f"A(z) {p} fázisban konfliktusos csoportok vannak: {a}, {b}")
    return plan
```

A.6. Forgatókönyv: `sumo_morl/scenario.py`

```
"""
A T-csomópont forgatókönyv: hálózat + jelzésterv + detektorok egy helyen.
"""
from __future__ import annotations

import os
from dataclasses import dataclass, field
from typing import List, Optional

from .detectors import DetectorInfo, detector_layout
from .intergreen import SignalPlan, build_signal_plan
from .network import APPROACHES, INCOMING_EDGES, TLS_ID

DATA_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "data")
DEFAULT_NET = os.path.join(DATA_DIR, "t_junction.net.xml")

#: Olvasható nevek a jelzőcsoportokhoz (bejövő sáv -> név).
GROUP_NAMES = {
    "W_C_0": "Ny_egyenes+jobb",
    "E_C_0": "K_egyenes",
    "E_C_1": "K_balra",
    "S_C_0": "D_bal+jobb",
}

#: A fázisok (jelzőcsoportok sávjai szerint) és neveik. Ezek pontosan a
#: konfliktusmentes klikkek, amelyeket a közbensőidő-mátrixból kapunk; itt csak
#: rögzítjük a sorrendjüket, hogy a fázisok sorszáma stabil legyen.
PHASE_LANES = [
    ["E_C_0", "W_C_0"],
```

```
    ["E_C_0", "E_C_1"],
    ["S_C_0"],
]
PHASE_NAMES = [
    "Főirány: nyugat-kelet egyenes (+ nyugatról jobbra)",
    "Keleti ág: egyenes + balra kanyarodó",
    "Mellékirány: déli ág (balra + jobbra)",
]


@dataclass
class TJunctionScenario:
    """Mindent tartalmaz, ami a szimulációhoz kell.

    Attributes:
        net_file: hálózat fájl.
        plan: a jelzésterv (fázisok, közbenső idők, átmenetek).
        detectors: a hurokdetektorok listája.
    """

    net_file: str = DEFAULT_NET
    yellow: int = 3
    red_yellow: int = 2
    plan: SignalPlan = field(init=False)
    detectors: List[DetectorInfo] = field(init=False)
    tls_id: str = TLS_ID
    approaches: tuple = APPROACHES

    def __post_init__(self):
        auto = build_signal_plan(self.net_file, self.tls_id, yellow=self.yellow,
                                 red_yellow=self.red_yellow, group_names=GROUP_NAMES)
        lane_to_group = {g.lane_id: i for i, g in enumerate(auto.groups)}
        phases = [sorted(lane_to_group[l] for l in lanes) for lanes in PHASE_LANES]
        if sorted(map(tuple, phases)) != sorted(map(tuple, auto.phases)):
            raise RuntimeError(f"A rögzített fázisok ({phases}) eltérnek a klikkektől ({auto.phases}).")
        self.plan = build_signal_plan(self.net_file, self.tls_id, phases=phases, yellow=self.yellow,
                                      red_yellow=self.red_yellow, group_names=GROUP_NAMES,
                                      phase_names=PHASE_NAMES)
        self.detectors = detector_layout(self.net_file, INCOMING_EDGES)

    def describe(self) -> str:
        out = [self.plan.describe(), "", "Hurokdetektorok:"]
        for d in self.detectors:
            out.append(f"  {d.id:<18} ág={d.approach} sáv={d.lane_id:<6} típus={d.kind:<8} poz={d.pos:6.1f} m")
        return "\n".join(out)
```

A.7. Hálózat generálása: `sumo_morl/network.py`

```
"""
A T-csomópont hálózatának (``.net.xml``) előállítása.

A hallgatóknak ehhez nem kell hozzányúlniuk: a csomag tartalmaz egy kész
hálózatot (``sumo_morl/data/t_junction.net.xml``). Ez a modul csak azért van
itt, hogy látható legyen, *hogyan* készült, és hogy szükség esetén (pl. más
ágszakasz-hossz vagy sebesség) újra lehessen generálni.

A csomópont felépítése (jobbkezes forgalom, észak felfelé)::

                  (nincs északi ág)
        W ---------------- C ---------------- E
```

```
     (nyugat)             |              (kelet)
                          |
                          S (dél)

* **Nyugati ág (W_C)**: 1 sáv, egyenesen keletre + jobbra délre (közös sáv).
* **Keleti ág (E_C)**: 2 sáv - ez a "jobb oldali" ág, amely ketté van véve:
    - 0. sáv (jobb oldali): egyenesen nyugatra,
    - 1. sáv (bal oldali): balra kanyarodó, délre.
* **Déli ág (S_C)**: 1 sáv, balra nyugatra + jobbra keletre (közös sáv).
"""
from __future__ import annotations

import os
import subprocess
import tempfile
from dataclasses import dataclass

from .sumo_utils import find_sumo_binary

# Ágak és mozgások nevei, a csomag többi része ezekre hivatkozik.
APPROACHES = ("W", "E", "S")
INCOMING_EDGES = {"W": "W_C", "E": "E_C", "S": "S_C"}
OUTGOING_EDGES = {"W": "C_W", "E": "C_E", "S": "C_S"}
TLS_ID = "C"

#: (honnan, hova) -> (bejövő él, sáv index)
MOVEMENTS = {
    ("W", "E"): ("W_C", 0),
    ("W", "S"): ("W_C", 0),
    ("E", "W"): ("E_C", 0),
    ("E", "S"): ("E_C", 1),
    ("S", "W"): ("S_C", 0),
    ("S", "E"): ("S_C", 0),
}


@dataclass
class NetworkSpec:
    """A T-csomópont geometriai paraméterei."""

    approach_length: float = 250.0   # ágak hossza [m]
    speed: float = 13.89             # megengedett sebesség [m/s] (50 km/h)


def _write(path: str, text: str) -> None:
    with open(path, "w", encoding="utf-8") as f:
        f.write(text)


def build_network(out_file: str, spec: NetworkSpec | None = None) -> str:
    """Legenerálja a T-csomópont ``.net.xml`` fájlját ``netconvert``-tel.

    Args:
        out_file: a kimeneti ``.net.xml`` útvonala.
        spec: geometriai paraméterek (alapértelmezés: :class:`NetworkSpec`).

    Returns:
        A létrehozott fájl útvonala.
    """
    spec = spec or NetworkSpec()
    L, v = spec.approach_length, spec.speed

    nodes = f"""<nodes>
    <node id="C" x="0" y="0" type="traffic_light"/>
    <node id="W" x="{-L}" y="0" type="priority"/>
```

```
    <node id="E" x="{L}" y="0" type="priority"/>
    <node id="S" x="0" y="{-L}" type="priority"/>
</nodes>
"""
    edges = f"""<edges>
    <edge id="W_C" from="W" to="C" numLanes="1" speed="{v}" priority="2"/>
    <edge id="C_W" from="C" to="W" numLanes="1" speed="{v}" priority="2"/>
    <edge id="E_C" from="E" to="C" numLanes="2" speed="{v}" priority="2"/>
    <edge id="C_E" from="C" to="E" numLanes="1" speed="{v}" priority="2"/>
    <edge id="S_C" from="S" to="C" numLanes="1" speed="{v}" priority="1"/>
    <edge id="C_S" from="C" to="S" numLanes="1" speed="{v}" priority="1"/>
</edges>
"""
    cons = """<connections>
    <connection from="W_C" to="C_E" fromLane="0" toLane="0"/>
    <connection from="W_C" to="C_S" fromLane="0" toLane="0"/>
    <connection from="E_C" to="C_W" fromLane="0" toLane="0"/>
    <connection from="E_C" to="C_S" fromLane="1" toLane="0"/>
    <connection from="S_C" to="C_W" fromLane="0" toLane="0"/>
    <connection from="S_C" to="C_E" fromLane="0" toLane="0"/>
</connections>
"""
    out_file = os.path.abspath(out_file)
    os.makedirs(os.path.dirname(out_file), exist_ok=True)
    with tempfile.TemporaryDirectory() as tmp:
        n, e, c = (os.path.join(tmp, x) for x in ("t.nod.xml", "t.edg.xml", "t.con.xml"))
        _write(n, nodes)
        _write(e, edges)
        _write(c, cons)
        cmd = [
            find_sumo_binary("netconvert"),
            "--node-files", n, "--edge-files", e, "--connection-files", c,
            "--output-file", out_file,
            "--no-turnarounds", "true",
            "--tls.default-type", "static",
            "--junctions.corner-detail", "8",
            "--no-warnings", "true",
        ]
        subprocess.run(cmd, check=True, capture_output=True)
    return out_file


if __name__ == "__main__":  # python -m sumo_morl.network
    here = os.path.join(os.path.dirname(__file__), "data", "t_junction.net.xml")
    print(build_network(here))
```

A.8. Forgalmi igény: `sumo_morl/traffic.py`

```
"""
Forgalmi igény (járműáramok) megadása és SUMO route fájl generálása.

Az igényt irányonként (honnan → hova) adjuk meg jármű/óra egységben. A
:meth:`Demand.random` minden epizódhoz véletlen, de reális igényt sorsol, így
az ágens változatos forgalmi helyzeteken tanul.
"""
from __future__ import annotations

import xml.etree.ElementTree as ET
from dataclasses import dataclass, field
from typing import Dict, Optional, Sequence, Tuple

import numpy as np

```

```python
from .network import INCOMING_EDGES, MOVEMENTS, OUTGOING_EDGES

Movement = Tuple[str, str]


@dataclass
class Demand:
    """Forgalmi igény.

    Attributes:
        flows: ``{(honnan, hova): jármű/óra}``, pl. ``{("W", "E"): 500}``.
        profile: időbeli szorzók (egyenlő hosszú szakaszokra osztva az
            epizódot), pl. ``(0.5, 1.0, 1.5, 1.0)`` egy csúcsórás hullámhoz.
            ``(1.0,)`` = állandó igény.
    """

    flows: Dict[Movement, float]
    profile: Sequence[float] = (1.0,)

    def __post_init__(self):
        for m in self.flows:
            if m not in MOVEMENTS:
                raise ValueError(f"Ismeretlen mozgás: {m}. Lehetséges: {list(MOVEMENTS)}")

    @property
    def total(self) -> float:
        return float(sum(self.flows.values()))

    def approach_volume(self, approach: str) -> float:
        return float(sum(v for (a, _), v in self.flows.items() if a == approach))

    @classmethod
    def constant(cls, main: float = 500, side: float = 200, left_share: float = 0.3,
                 right_share: float = 0.2) -> "Demand":
        """Egyszerű, állandó igény a főirányon (ny-k) és a mellékirányon (dél).

        Args:
            main: a főirány egy irányának forgalma [jármű/óra].
            side: a déli mellékág forgalma [jármű/óra].
            left_share: a keletről balra (délre) kanyarodók aránya.
            right_share: a nyugatról jobbra (délre) kanyarodók aránya.
        """
        return cls({
            ("W", "E"): main * (1 - right_share), ("W", "S"): main * right_share,
            ("E", "W"): main * (1 - left_share), ("E", "S"): main * left_share,
            ("S", "W"): side * 0.5, ("S", "E"): side * 0.5,
        })

    @classmethod
    def random(cls, rng: np.random.Generator, main_range=(200, 700), side_range=(80, 400),
               turn_range=(0.1, 0.45), profile: Optional[Sequence[float]] = None) -> "Demand":
        """Véletlen igény sorsolása (epizódonként más-más)."""
        mw, me = rng.uniform(*main_range, size=2)
        side = rng.uniform(*side_range)
        rs, ls, ss = rng.uniform(*turn_range, size=3)
        flows = {
            ("W", "E"): mw * (1 - rs), ("W", "S"): mw * rs,
            ("E", "W"): me * (1 - ls), ("E", "S"): me * ls,
            ("S", "W"): side * (1 - ss), ("S", "E"): side * ss,
        }
        return cls(flows, tuple(profile) if profile is not None else (1.0,))

    def scaled(self, factor: float) -> "Demand":
        return Demand({m: v * factor for m, v in self.flows.items()}, self.profile)

```

```
    def write_routes(self, out_file: str, duration: int, seed: int = 0) -> str:
        """SUMO ``.rou.xml`` írása ``<flow>`` elemekkel (Poisson-szerű érkezés)."""
        root = ET.Element("routes")
        ET.SubElement(root, "vType", id="car", vClass="passenger", length="4.5", minGap="2.5",
                      accel="2.6", decel="4.5", sigma="0.5", maxSpeed="16.67")
        for (a, b) in self.flows:
            ET.SubElement(root, "route", id=f"r_{a}{b}", edges=f"{INCOMING_EDGES[a]} {
    OUTGOING_EDGES[b]}")
        n_seg = len(self.profile)
        seg = duration / n_seg
        elems = []
        for k, mult in enumerate(self.profile):
            for (a, b), vph in self.flows.items():
                rate = vph * mult
                if rate <= 0:
                    continue
                elems.append((k * seg, dict(
                    id=f"f_{a}{b}_{k}", type="car", route=f"r_{a}{b}", begin=f"{k * seg:.0f}",
                    end=f"{(k + 1) * seg:.0f}", period=f"exp({rate / 3600.0:.6f})",
                    departLane="best", departSpeed="max")))
        for _, attrs in sorted(elems, key=lambda e: e[0]):
            ET.SubElement(root, "flow", **attrs)
        ET.indent(root)
        ET.ElementTree(root).write(out_file, encoding="utf-8", xml_declaration=True)
        return out_file

    def __str__(self) -> str:
        parts = ", ".join(f"{a}->{b}: {v:.0f}" for (a, b), v in self.flows.items())
        return f"Demand({parts} jármű/óra, profil={tuple(self.profile)})"
```

A.9. Összehasonlító vezérlők: `sumo_morl/policies.py`

```
"""
Egyszerű, nem tanuló vezérlők összehasonlítási alapnak (baseline).

Mindegyik csak a hurokdetektorok adatait és a vezérlő saját állapotát
használja (``info``), ahogy egy valódi forgalomirányító berendezés is.

Használat::

    policy = FixedTimePolicy(env, green_times=[30, 10, 15])
    obs, info = env.reset()
    policy.reset()
    action = policy(obs, info)
"""
from __future__ import annotations

from typing import Dict, Optional, Sequence

import numpy as np


class Policy:
    """Alap osztály: ``policy(obs, info) -> akció``."""

    def __init__(self, env):
        self.env = env.unwrapped if hasattr(env, "unwrapped") else env

    def reset(self) -> None:
        pass

    def __call__(self, obs, info) -> int:  # pragma: no cover - absztrakt
        raise NotImplementedError
```

```python


class RandomPolicy(Policy):
    """Véletlen fázisválasztás."""

    def __init__(self, env, seed: Optional[int] = None):
        super().__init__(env)
        self.rng = np.random.default_rng(seed)

    def __call__(self, obs, info) -> int:
        return int(self.rng.integers(self.env.n_phases))


class FixedTimePolicy(Policy):
    """Fix idejű (ciklikus) vezérlés: minden fázis adott ideig zöld.

    Args:
        green_times: zöldidők fázisonként [s]. Mivel a döntések
            ``decision_interval`` lépésekben jönnek, a tényleges zöldidő ennek
            többszörösére kerekedik fel.
    """

    def __init__(self, env, green_times: Sequence[float] = (30, 15, 15)):
        super().__init__(env)
        if len(green_times) != self.env.n_phases:
            raise ValueError(f"{self.env.n_phases} zöldidő kell (fázisonként egy).")
        self.green_times = list(green_times)

    def __call__(self, obs, info) -> int:
        p = info["phase"]
        if info["time_in_phase"] >= self.green_times[p]:
            return (p + 1) % self.env.n_phases
        return p


class ActuatedPolicy(Policy):
    """Járműkövető (forgalomfüggő) vezérlés időrés-figyeléssel.

    A zöld addig hosszabbodik, amíg az aktuális fázis sávjainak előjelző
    (``advance``) hurkain ``gap`` másodpercen belül volt érzékelés, de legfeljebb
    ``max_green`` másodpercig. Utána a következő olyan fázisra vált, amelynek a
    sávjaiban van igény (stopvonali vagy előjelző hurok foglalt / számlált).
    """

    def __init__(self, env, max_green: float = 45, gap: float = 3.0):
        super().__init__(env)
        self.max_green = max_green
        self.gap = gap

    def _lane_demand(self, snap, lane: str) -> bool:
        m = snap.select(lane_id=lane) & (snap.select(kind="stopline") | snap.select(kind="advance")
    )
        return bool((snap.occupancy[m] > 0).any() or (snap.count[m] > 0).any()
                    or (snap.gap[m] < self.gap).any())

    def __call__(self, obs, info) -> int:
        snap = info["detectors"]
        p = info["phase"]
        n = self.env.n_phases
        lanes = self.env.plan.phase_lanes(p)
        adv = np.zeros(len(snap.detectors), dtype=bool)
        for l in lanes:
            adv |= snap.select(lane_id=l, kind="advance")
        still_coming = bool((snap.gap[adv] < self.gap).any())
        if still_coming and info["time_in_phase"] < self.max_green:
```

```
            return p
        for k in range(1, n + 1):
            q = (p + k) % n
            if q == p:
                return p
            if any(self._lane_demand(snap, l) for l in self.env.plan.phase_lanes(q)):
                return q
        return p


class LongestQueuePolicy(Policy):
    """Mohó vezérlő: a legnagyobb becsült igényű fázist választja.

    Az igény a fázis sávjainak stopvonali + előjelző hurok foglaltsága, valamint
    a sávok ágainak bemenet-kimenet járműszám-becslése alapján számolódik.
    """

    def __init__(self, env, min_hold: float = 10.0):
        super().__init__(env)
        self.min_hold = min_hold

    def phase_scores(self, snap) -> np.ndarray:
        scores = np.zeros(self.env.n_phases)
        for p in range(self.env.n_phases):
            for l in self.env.plan.phase_lanes(p):
                m = snap.select(lane_id=l) & ~snap.select(kind="entry")
                scores[p] += snap.occupancy[m].sum()
                app = next(d.approach for d in snap.detectors if d.lane_id == l)
                n_lanes = len({d.lane_id for d in snap.detectors if d.approach == app})
                scores[p] += 0.1 * snap.approach_vehicles[app] / n_lanes
        return scores

    def __call__(self, obs, info) -> int:
        if info["time_in_phase"] < self.min_hold:
            return info["phase"]
        return int(np.argmax(self.phase_scores(info["detectors"])))
```

A.10. MORL segédfüggvények: `sumo_morl/morl.py`

```
"""
MORL segédfüggvények: skalarizálás, súlyvektorok, Pareto-front, hipertérfogat,
epizód futtatás és kiértékelés, ábrázolás.
"""
from __future__ import annotations

import itertools
from dataclasses import dataclass, field
from typing import Callable, Dict, List, Optional, Sequence

import gymnasium as gym
import numpy as np


# ============================================================================ wrapper
class LinearScalarization(gym.Wrapper):
    """A vektor-jutalmat súlyozott összeggé alakítja: ``r = w · r_vec``.

    Így bármely egycélú RL könyvtár (pl. Stable-Baselines3) használható. Az
    eredeti vektor az ``info["vector_reward"]`` alatt marad elérhető.

    Args:
        env: a többcélú környezet.
        weights: súlyvektor (célfüggvényenként egy szám, tipikusan összegük 1).
```

```
    """

    def __init__(self, env: gym.Env, weights: Sequence[float]):
        super().__init__(env)
        self.weights = np.asarray(weights, dtype=np.float32)
        if self.weights.shape != (env.unwrapped.reward_dim,):
            raise ValueError(f"{env.unwrapped.reward_dim} súly kell, kaptam: {self.weights.shape}")

    def set_weights(self, weights: Sequence[float]) -> None:
        self.weights = np.asarray(weights, dtype=np.float32)

    def step(self, action):
        obs, r, term, trunc, info = self.env.step(action)
        info = dict(info)
        info["vector_reward"] = r
        return obs, float(np.dot(self.weights, r)), term, trunc, info


class WeightConditioned(gym.ObservationWrapper):
    """A súlyvektort hozzáfűzi a megfigyeléshez (súly-kondicionált ágensekhez).

    Minden ``reset``-kor új súlyt sorsol (vagy fixet használ), és a jutalmat
    is ezzel skalarizálja. Egyetlen hálóval így a teljes Pareto-front tanulható.
    """

    def __init__(self, env: gym.Env, weights: Optional[Sequence[float]] = None, seed: Optional[int]
     = None):
        super().__init__(env)
        self.k = env.unwrapped.reward_dim
        self.fixed = None if weights is None else np.asarray(weights, dtype=np.float32)
        self.rng = np.random.default_rng(seed)
        self.weights = self.fixed if self.fixed is not None else random_weights(self.k, 1, self.rng
    )[0]
        low = np.concatenate([env.observation_space.low, np.zeros(self.k)]).astype(np.float32)
        high = np.concatenate([env.observation_space.high, np.ones(self.k)]).astype(np.float32)
        self.observation_space = gym.spaces.Box(low, high, dtype=np.float32)

    def reset(self, **kw):
        if self.fixed is None:
            self.weights = random_weights(self.k, 1, self.rng)[0]
        return super().reset(**kw)

    def observation(self, obs):
        return np.concatenate([obs, self.weights]).astype(np.float32)

    def step(self, action):
        obs, r, term, trunc, info = self.env.step(action)
        info = dict(info)
        info["vector_reward"] = r
        return self.observation(obs), float(np.dot(self.weights, r)), term, trunc, info


# ============================================================================ súlyok
def equally_spaced_weights(n_objectives: int, n_divisions: int) -> np.ndarray:
    """Egyenletes rács a súly-szimplexen (összeg = 1).

    Pl. ``equally_spaced_weights(2, 4)`` ->
    ``[[0,1], [.25,.75], [.5,.5], [.75,.25], [1,0]]``.
    """
    pts = []
    for c in itertools.product(range(n_divisions + 1), repeat=n_objectives):
        if sum(c) == n_divisions:
            pts.append(np.array(c, dtype=float) / n_divisions)
    return np.array(sorted(pts, key=lambda p: tuple(p)))

```

```python

def random_weights(n_objectives: int, n: int = 1, rng: Optional[np.random.Generator] = None) -> np.ndarray:
    """Egyenletes eloszlású véletlen súlyvektorok a szimplexen (Dirichlet(1))."""
    rng = rng or np.random.default_rng()
    return rng.dirichlet(np.ones(n_objectives), size=n).astype(np.float32)


# ============================================================================ Pareto
def pareto_mask(points: np.ndarray) -> np.ndarray:
    """Igaz azokra a pontokra, amelyek nem dominált (Pareto-optimális) pontok.

    Konvenció: minden célt **maximalizálunk** (a környezet jutalmai ilyenek).
    """
    pts = np.asarray(points, dtype=float)
    n = len(pts)
    mask = np.ones(n, dtype=bool)
    for i in range(n):
        if not mask[i]:
            continue
        dominated = np.all(pts >= pts[i], axis=1) & np.any(pts > pts[i], axis=1)
        if dominated.any():
            mask[i] = False
    return mask


def pareto_front(points: np.ndarray) -> np.ndarray:
    """A nem dominált pontok (lásd :func:`pareto_mask`)."""
    pts = np.asarray(points, dtype=float)
    return pts[pareto_mask(pts)]


def hypervolume(points: np.ndarray, ref: Sequence[float]) -> float:
    """Hipertérfogat (maximalizálás, ``ref`` = minden pontnál rosszabb referencia).

    Pontos, rekurzív szeletelős algoritmus; kis pontszámra (< néhány száz) és
    2-4 célra bőven elég gyors.
    """
    ref = np.asarray(ref, dtype=float)
    pts = np.asarray(points, dtype=float)
    pts = pts[np.all(pts > ref, axis=1)]
    if len(pts) == 0:
        return 0.0
    pts = pareto_front(pts)
    return float(_hv(pts - ref))


def _hv(p: np.ndarray) -> float:
    if p.shape[1] == 1:
        return float(p.max())
    p = p[np.argsort(-p[:, -1])]
    vol, prev = 0.0, 0.0
    # szeletek az utolsó koordináta mentén, alulról felfelé
    levels = np.unique(p[:, -1])
    for lvl in levels:
        sub = p[p[:, -1] >= lvl][:, :-1]
        vol += _hv(pareto_front(sub)) * (lvl - prev)
        prev = lvl
    return vol


# ============================================================================ futtatás
@dataclass
class EpisodeResult:
    vector_return: np.ndarray
```

```
    objective_names: Sequence[str]
    steps: int
    ground_truth: Dict[str, float] = field(default_factory=dict)
    history: List[Dict] = field(default_factory=list)

    def as_dict(self) -> Dict[str, float]:
        d = {n: float(v) for n, v in zip(self.objective_names, self.vector_return)}
        d.update({f"gt_{k}": v for k, v in self.ground_truth.items()})
        return d


def run_episode(env: gym.Env, policy: Callable, seed: Optional[int] = None, record: bool = False,
                gamma: float = 1.0) -> EpisodeResult:
    """Egy teljes epizód lefuttatása egy vezérlővel.

    Args:
        env: a (nem skalarizált) többcélú környezet.
        policy: ``policy(obs, info) -> akció`` függvény (vagy :class:`~sumo_morl.policies.Policy`).
            Stable-Baselines3 modellhez lásd :func:`sb3_policy`.
        seed: a reset magja (azonos mag = azonos forgalom).
        record: lépésenkénti napló (``history``) gyűjtése.
        gamma: diszkont (1.0 = egyszerű összeg).
    """
    if hasattr(policy, "reset"):
        policy.reset()
    obs, info = env.reset(seed=seed)
    unwrapped = env.unwrapped
    ret = np.zeros(unwrapped.reward_dim)
    disc, steps, hist = 1.0, 0, []
    done = False
    while not done:
        a = policy(obs, info)
        obs, r, term, trunc, info = env.step(a)
        r_vec = np.asarray(info.get("vector_reward", r), dtype=float)
        ret += disc * r_vec
        disc *= gamma
        steps += 1
        if record:
            hist.append({"time": info["time"], "action": int(a), "phase": info["phase"],
                         **{f"r_{k}": v for k, v in info["objectives"].items()},
                         **{f"veh_{k}": v for k, v in info["detectors"].approach_vehicles.items()
    }})
        done = term or trunc
    return EpisodeResult(ret, unwrapped.objective_names, steps, dict(info.get("ground_truth", {})),
     hist)


def evaluate_policy(env: gym.Env, policy: Callable, n_episodes: int = 3,
                    seeds: Optional[Sequence[int]] = None) -> Dict[str, np.ndarray]:
    """Több epizódos kiértékelés. Visszaadja a vektor-hozamok átlagát és szórását.

    Azonos ``seeds`` listával a különböző vezérlők azonos forgalmon hasonlíthatók össze.
    """
    seeds = list(seeds) if seeds is not None else list(range(n_episodes))
    results = [run_episode(env, policy, seed=s) for s in seeds]
    R = np.array([r.vector_return for r in results])
    gt_keys = results[0].ground_truth.keys()
    gt = {k: float(np.nanmean([r.ground_truth[k] for r in results])) for k in gt_keys}
    return {"mean": R.mean(axis=0), "std": R.std(axis=0), "returns": R, "ground_truth": gt,
            "objective_names": list(env.unwrapped.objective_names)}


def sb3_policy(model, weights: Optional[Sequence[float]] = None, deterministic: bool = True) ->
    Callable:
    """Stable-Baselines3 modellből ``policy(obs, info)`` függvényt készít.
```

```
214
215     Ha a modellt :class:`WeightConditioned` környezeten tanítottuk, add meg a
216     ``weights`` súlyt is, ezt fűzzük a megfigyeléshez.
217     """
218     w = None if weights is None else np.asarray(weights, dtype=np.float32)
219
220     def _pol(obs, info):
221         o = obs if w is None else np.concatenate([obs, w]).astype(np.float32)
222         a, _ = model.predict(o, deterministic=deterministic)
223         return int(a)
224
225     return _pol
226
227
228 # ============================================================================ ábra
229 def plot_pareto(points: np.ndarray, objective_names: Sequence[str], labels: Optional[Sequence[str]]
        = None,
230                 ax=None, highlight_front: bool = True, title: str = "Pareto-front"):
231     """2D Pareto-ábra (több cél esetén minden célpárra külön ábra).
232
233     Returns:
234         A matplotlib ``Figure``.
235     """
236     import matplotlib.pyplot as plt
237
238     pts = np.asarray(points, dtype=float)
239     k = pts.shape[1]
240     pairs = list(itertools.combinations(range(k), 2))
241     if ax is None:
242         fig, axes = plt.subplots(1, len(pairs), figsize=(5 * len(pairs), 4.2), squeeze=False)
243         axes = axes[0]
244     else:
245         axes = [ax]
246         fig = ax.figure
247     mask = pareto_mask(pts) if highlight_front else np.zeros(len(pts), bool)
248     for a, (i, j) in zip(axes, pairs):
249         a.scatter(pts[~mask, i], pts[~mask, j], c="0.6", label="dominált")
250         a.scatter(pts[mask, i], pts[mask, j], c="C3", label="Pareto-optimális")
251         if labels is not None:
252             for p, lab in zip(pts, labels):
253                 a.annotate(str(lab), (p[i], p[j]), fontsize=7, xytext=(3, 3), textcoords="offset
    points")
254         a.set_xlabel(objective_names[i])
255         a.set_ylabel(objective_names[j])
256         a.grid(alpha=0.3)
257     axes[0].legend()
258     fig.suptitle(title)
259     fig.tight_layout()
260     return fig
```

A.11. Ábrák: `sumo_morl/visualize.py`

```
1  """
2  Ábrák a forgatókönyvről és az epizódokról (matplotlib).
3  """
4  from __future__ import annotations
5
6  import xml.etree.ElementTree as ET
7  from typing import Dict, List, Optional, Sequence
8
9  import numpy as np
10
11 STATE_COLORS = {"G": "#2ca02c", "g": "#2ca02c", "y": "#ffbf00", "u": "#ff7f0e", "r": "#d62728"}
```

```python


def _shapes(net_file: str):
    root = ET.parse(net_file).getroot()
    lanes, internal = {}, {}
    for e in root.findall("edge"):
        for l in e.findall("lane"):
            pts = np.array([list(map(float, p.split(","))) for p in l.get("shape").split()])
            (internal if e.get("function") == "internal" else lanes)[l.get("id")] = pts
    via = {}
    for c in root.findall("connection"):
        if c.get("linkIndex") is not None and c.get("via"):
            via[int(c.get("linkIndex"))] = c.get("via")
    # összefűzött belső útvonalak (pl. :C_1_0 + :C_6_0)
    paths = {}
    for idx, v in via.items():
        pts = internal[v]
        changed = True
        while changed:
            changed = False
            for lid, p2 in internal.items():
                if lid != v and np.linalg.norm(p2[0] - pts[-1]) < 0.3 and lid not in [x for x in via.values()]:
                    pts = np.vstack([pts, p2[1:]])
                    changed = True
                    break
        paths[idx] = pts
    return lanes, paths


def _draw_lanes(ax, lanes, color="0.8"):
    from matplotlib.patches import Polygon
    from shapely.geometry import LineString

    for pts in lanes.values():
        poly = LineString(pts).buffer(1.55, cap_style=2)
        ax.add_patch(Polygon(np.array(poly.exterior.coords), closed=True, fc=color, ec="0.6", lw=0.3,
                             zorder=1))


def plot_scenario(scenario=None, show_detectors: bool = True, figsize=(12, 5.5)):
    """A csomópont rajza két nézetben (teljes + nagyított): sávok, mozgások, hurokdetektorok."""
    import matplotlib.pyplot as plt
    from .scenario import TJunctionScenario

    scenario = scenario or TJunctionScenario()
    lanes, paths = _shapes(scenario.net_file)
    cx, cy = np.mean([p.mean(axis=0) for p in paths.values()], axis=0)
    fig, (ax0, ax1) = plt.subplots(1, 2, figsize=figsize, gridspec_kw={"width_ratios": [1.2, 1]})
    colors = {"entry": "C0", "advance": "C1", "stopline": "C3"}
    kind_hu = {"entry": "belépő (entry)", "advance": "előjelző (advance)", "stopline": "stopvonali (stopline)"}
    for ax, zoom in ((ax0, None), (ax1, 55.0)):
        _draw_lanes(ax, lanes)
        for pts in paths.values():
            ax.plot(pts[:, 0], pts[:, 1], color="0.35", lw=1, zorder=3)
            ax.annotate("", xy=pts[-1], xytext=pts[-2], arrowprops=dict(arrowstyle="-|>", color="0.35"))
        if show_detectors:
            seen = set()
            for d in scenario.detectors:
                pts = lanes[d.lane_id]
                seg = np.cumsum(np.r_[0, np.linalg.norm(np.diff(pts, axis=0), axis=1)])
```

```
                x, y = np.interp(d.pos, seg, pts[:, 0]), np.interp(d.pos, seg, pts[:, 1])
                ax.plot(x, y, "s", color=colors[d.kind], ms=5 if zoom is None else 8, zorder=5,
                        label=kind_hu[d.kind] if d.kind not in seen else None)
                seen.add(d.kind)
                if zoom is not None and abs(x - cx) < zoom and abs(y - cy) < zoom:
                    ax.annotate(d.id, (x, y), fontsize=7, xytext=(5, 5), textcoords="offset points")
        ax.set_aspect("equal")
        if zoom is not None:
            ax.set_xlim(cx - zoom, cx + zoom)
            ax.set_ylim(cy - zoom, cy + zoom * 0.5)
            ax.set_title("Nagyítás a csomópontra")
        else:
            ax.set_title("T-csomópont (teljes hálózat)")
            for txt, lid, end in (("Nyugat (W)", "W_C_0", 0), ("Kelet (E)", "E_C_0", 0), ("Dél (S)", "S_C_0", 0)):
                ax.annotate(txt, lanes[lid][end], fontsize=10, fontweight="bold", xytext=(0, 8),
                            textcoords="offset points", ha="center")
            ax.legend(loc="lower left", title="hurokdetektor", fontsize=8)
            ax.set_xlabel("x [m]")
            ax.set_ylabel("y [m]")
    fig.tight_layout()
    return fig


def plot_phases(scenario=None, figsize=None):
    """Fázisonként egy ábra: melyik mozgás kap zöldet."""
    import matplotlib.pyplot as plt
    from .scenario import TJunctionScenario

    scenario = scenario or TJunctionScenario()
    plan = scenario.plan
    lanes, paths = _shapes(scenario.net_file)
    n = plan.n_phases
    fig, axes = plt.subplots(1, n, figsize=figsize or (4 * n, 4))
    cx, cy = np.mean([p.mean(axis=0) for p in paths.values()], axis=0)
    for p, ax in enumerate(np.atleast_1d(axes)):
        state = plan.green_state(p)
        for pts in lanes.values():
            ax.plot(pts[:, 0], pts[:, 1], color="0.85", lw=7, solid_capstyle="butt")
        for idx, pts in paths.items():
            green = state[idx] in "Gg"
            col = STATE_COLORS["G"] if green else "0.8"
            ax.plot(pts[:, 0], pts[:, 1], color=col, lw=2.5 if green else 1, zorder=3 if green else 2)
            ax.annotate("", xy=pts[-1], xytext=pts[-2], arrowprops=dict(arrowstyle="-|>", color=col))
        ax.set_xlim(cx - 30, cx + 30)
        ax.set_ylim(cy - 30, cy + 20)
        ax.set_aspect("equal")
        ax.set_xticks([])
        ax.set_yticks([])
        name = plan.phase_names[p] if p < len(plan.phase_names) else ""
        ax.set_title(f"Fázis {p}\n{name}", fontsize=9)
    fig.tight_layout()
    return fig


def plot_intergreen(plan, ax=None):
    """A jelzőcsoportok közötti közbenső idő mátrix hőtérképe."""
    import matplotlib.pyplot as plt

    if ax is None:
        _, ax = plt.subplots(figsize=(5, 4))
    m = plan.intergreen
```

```
    ax.imshow(np.where(np.isnan(m), -1, m), cmap="Oranges", vmin=0)
    names = [g.name for g in plan.groups]
    ax.set_xticks(range(len(names)), names, rotation=45, ha="right")
    ax.set_yticks(range(len(names)), names)
    for i in range(len(names)):
        for j in range(len(names)):
            ax.text(j, i, "-" if np.isnan(m[i, j]) else int(m[i, j]), ha="center", va="center")
    ax.set_xlabel("belépő (zöldet kap)")
    ax.set_ylabel("kiürítő (zöldje véget ér)")
    ax.set_title("Közbenső idők [s]")
    return ax


def plot_episode(history: Sequence[Dict], phase_names: Optional[List[str]] = None):
    """Egy rögzített epizód (``run_episode(..., record=True).history``) ábrája:
    fázisok az időben, becsült járműszám ágankként és a jutalomkomponensek."""
    import matplotlib.pyplot as plt

    t = np.array([h["time"] for h in history])
    fig, axes = plt.subplots(3, 1, figsize=(10, 7), sharex=True)
    axes[0].step(t, [h["phase"] for h in history], where="pre")
    axes[0].set_ylabel("fázis")
    if phase_names:
        axes[0].set_yticks(range(len(phase_names)))
    for k in [k for k in history[0] if k.startswith("veh_")]:
        axes[1].plot(t, [h[k] for h in history], label=k[4:])
    axes[1].set_ylabel("becsült jármű/ág")
    axes[1].legend()
    for k in [k for k in history[0] if k.startswith("r_")]:
        axes[2].plot(t, [h[k] for h in history], label=k[2:])
    axes[2].set_ylabel("jutalom")
    axes[2].set_xlabel("idő [s]")
    axes[2].legend()
    fig.tight_layout()
    return fig


def plot_signal_timeline(signal_log, plan, t_from: int = 0, t_to: Optional[int] = None, ax=None):
    """Klasszikus jelzésterv-ábra (sávos diagram): jelzőcsoportonként a jelzés
    színe az időben. ``signal_log`` = ``env.signal_log`` (``record_signals=True``)."""
    import matplotlib.pyplot as plt

    log = [(t, s) for t, s in signal_log if t >= t_from and (t_to is None or t <= t_to)]
    if ax is None:
        _, ax = plt.subplots(figsize=(12, 0.6 * len(plan.groups) + 1.2))
    for gi, g in enumerate(plan.groups):
        li = g.links[0]
        for t, s in log:
            ax.barh(gi, 1, left=t - 1, color=STATE_COLORS.get(s[li], "0.5"), edgecolor="none",
    height=0.6)
    ax.set_yticks(range(len(plan.groups)), [g.name for g in plan.groups])
    ax.invert_yaxis()
    ax.set_xlabel("idő [s]")
    ax.set_title("Jelzésterv (zöld / sárga / piros-sárga / piros)")
    ax.set_xlim(log[0][0] - 1 if log else 0, log[-1][0] if log else 1)
    ax.figure.tight_layout()
    return ax
```

A.12. SUMO segédfüggvények: `sumo_morl/sumo_utils.py`

```
"""SUMO-val kapcsolatos apró segédfüggvények (bináris keresés, indítás)."""
from __future__ import annotations

```

```python
import os
import shutil


def sumo_home() -> str | None:
    """Visszaadja a SUMO telepítési könyvtárát (``SUMO_HOME``), ha megtalálható."""
    if os.environ.get("SUMO_HOME"):
        return os.environ["SUMO_HOME"]
    try:  # pip install eclipse-sumo
        import sumo  # type: ignore

        return sumo.SUMO_HOME
    except Exception:
        return None


def find_sumo_binary(name: str = "sumo") -> str:
    """Megkeresi a ``sumo`` / ``sumo-gui`` / ``netconvert`` futtatható fájlt."""
    home = sumo_home()
    if home:
        for cand in (os.path.join(home, "bin", name), os.path.join(home, "bin", name + ".exe")):
            if os.path.isfile(cand):
                return cand
    found = shutil.which(name)
    if found:
        return found
    raise FileNotFoundError(
        f"Nem található a(z) '{name}' program. Telepítsd a SUMO-t "
        "(pl. `pip install eclipse-sumo`), vagy állítsd be a SUMO_HOME változót."
    )
```

A.13. Közbensőidő-számítás: geometria: `geometry_utils.py`

```python
import numpy as np
import math
from scipy.interpolate import splprep, splev
from shapely.geometry import LineString, Point, MultiPoint
import matplotlib.pyplot as plt

def remove_close_points(coords, min_dist=0.8):
    """
    Removes consecutive points that are closer than min_dist.

    Args:
        coords (list): List of (x, y) tuples.
        min_dist (float): Minimum distance threshold.

    Returns:
        list: Filtered list of coordinates.
    """
    if len(coords) < 2: return coords
    clean = [coords[0]]
    for i in range(1, len(coords)):
        if np.linalg.norm(np.array(coords[i]) - np.array(clean[-1])) > min_dist:
            clean.append(coords[i])
    if np.linalg.norm(np.array(coords[-1]) - np.array(clean[-1])) > 0.001:
        if np.linalg.norm(np.array(coords[-1]) - np.array(clean[-1])) < min_dist:
            clean[-1] = coords[-1]
        else:
            clean.append(coords[-1])
    return clean


```

```python
def merge_segments_average(segments):
    """
    Merges connected lane segments into single continuous paths.
    Useful for handling SUMO's internal lane segmentation.

    Args:
        segments (list): List of segment dictionaries.

    Returns:
        list: List of merged segment dictionaries.
    """
    merged = [s.copy() for s in segments]
    something_changed = True
    TOLERANCE = 0.5
    while something_changed:
        something_changed = False
        i = 0
        while i < len(merged):
            j = 0
            merged_in_this_step = False
            while j < len(merged):
                if i == j: j += 1; continue
                seg_a = merged[i];
                seg_b = merged[j]
                end_a = np.array(seg_a['points'][-1]);
                start_b = np.array(seg_b['points'][0])
                if np.linalg.norm(end_a - start_b) < TOLERANCE:
                    mid_point = (end_a + start_b) / 2.0
                    new_points = seg_a['points'][:-1] + [tuple(mid_point)] + seg_b['points'][1:]
                    new_id = f"{seg_a['id']} + {seg_b['id']}"
                    merged[i]['points'] = new_points;
                    merged[i]['id'] = new_id
                    del merged[j]
                    something_changed = True;
                    merged_in_this_step = True;
                    break
                j += 1
            if not merged_in_this_step: i += 1
    return merged


def clip_polyline_by_distance(coords, keep_length=25.0):
    """
    Clips a polyline to keep only the last 'keep_length' meters.

    Args:
        coords (list): List of (x, y) tuples.
        keep_length (float): Length to keep from the end.

    Returns:
        list: Clipped coordinates.
    """
    if not coords or len(coords) < 2: return coords
    total_dist = 0.0
    for i in range(len(coords) - 1, 0, -1):
        p1 = np.array(coords[i]);
        p2 = np.array(coords[i - 1])
        dist = np.linalg.norm(p1 - p2)
        if total_dist + dist >= keep_length:
            remaining = keep_length - total_dist
            ratio = remaining / dist
            new_start_x = p1[0] + (p2[0] - p1[0]) * ratio
            new_start_y = p1[1] + (p2[1] - p1[1]) * ratio
            return [(new_start_x, new_start_y)] + coords[i:]
        total_dist += dist
```

```
    return coords


def calculate_radius(tck, u_val):
    """
    Calculates the curvature radius at a specific point on a spline.
    """
    dx, dy = splev(u_val, tck, der=1)
    ddx, ddy = splev(u_val, tck, der=2)
    num = np.abs(dx * ddy - dy * ddx)
    den = np.power(dx ** 2 + dy ** 2, 1.5)
    if num < 1e-6: return float('inf'), (0, 0), (0, 0)
    r = 1.0 / (num / den)
    l = np.sqrt(dx ** 2 + dy ** 2)
    s = np.sign(dx * ddy - dy * ddx)
    return r, (-dy * s / l, dx * s / l)


def get_geometric_radius(p1, p2, p3):
    """
    Calculates the radius of the circumscribed circle of three points.
    """
    x1, y1 = p1;
    x2, y2 = p2;
    x3, y3 = p3
    a = np.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
    b = np.sqrt((x3 - x2) ** 2 + (y3 - y2) ** 2)
    c = np.sqrt((x3 - x1) ** 2 + (y3 - y1) ** 2)
    s = (a + b + c) / 2
    area_sq = s * (s - a) * (s - b) * (s - c)
    if area_sq <= 1e-9: return float('inf')
    area = np.sqrt(area_sq)
    R = (a * b * c) / (4 * area)
    return R


def get_velocity(radius, mode='clearing'):
    """
    Estimates vehicle velocity based on turning radius.

    Args:
        radius (float): Turning radius in meters.
        mode (str): 'clearing' or 'entering'.

    Returns:
        float: Velocity in m/s.
    """
    if radius <= 6.0:
        v = 5.0
    elif 6.0 < radius < 25.0:
        v = math.sqrt(4 * radius)
    else:
        v = 10.0
    if mode == 'entering' and radius >= 25.0: v = 13.89
    return v


def calculate_intergreen_time(dist_clearing, dist_entering, r_clearing, r_entering):
    """
    Calculates the required intergreen time (K value) between two conflicting streams.
    Based on the German RiLSA or similar guidelines.

    Args:
        dist_clearing (float): Distance from conflict point to clearing line.
        dist_entering (float): Distance from stop line to conflict point.
```

```
        r_clearing (float): Radius of clearing vehicle path.
        r_entering (float): Radius of entering vehicle path.

    Returns:
        int: Calculated intergreen time in seconds (rounded up).
    """
    A = 3.0;
    L_VEHICLE = 6.0
    v_clearing = get_velocity(r_clearing, 'clearing')
    v_entering = get_velocity(r_entering, 'entering')
    U = (dist_clearing + L_VEHICLE) / v_clearing
    B = dist_entering / v_entering
    K = A + U - B
    return max(0, math.ceil(K))


def get_short_name(raw_id):
    """
    Generates a shorter, more readable name for a lane ID.
    """
    if " + " in raw_id:
        parts = raw_id.split(" + ")
        try:
            s_id = parts[0].split("_")[-2:];
            e_id = parts[-1].split("_")[-2:]
            return f"{s_id[0]}_{s_id[1]}->{e_id[0]}_{e_id[1]}"
        except:
            return raw_id
    else:
        try:
            s_id = raw_id.split("_")[-2:];
            return f"{s_id[0]}_{s_id[1]}"
        except:
            return raw_id


def calculate_junction_data(jdata):
    """
    Main function to process junction data and calculate conflict matrices.

    1. Merges segments.
    2. Smooths paths using splines.
    3. Calculates radii and velocities.
    4. Finds intersection points between all pairs of paths.
    5. Calculates intergreen times for all conflicts.

    Args:
        jdata (dict): Raw junction data from parser.

    Returns:
        dict: Processed data including lines, matrix, intersections, and bounds.
    """
    merged_paths = merge_segments_average(jdata['segments'])

    base_colors = plt.cm.tab20(np.linspace(0, 1, len(merged_paths)))
    darker_colors = []
    for c in base_colors:
        darker_colors.append((c[0] * 0.7, c[1] * 0.7, c[2] * 0.7, 1.0))
    colors = darker_colors

    lines_data = [];
    all_x, all_y = [], []
    if 'shape' in jdata and jdata['shape']:
        all_x.extend([p[0] for p in jdata['shape']]);
        all_y.extend([p[1] for p in jdata['shape']])
```

```
    for idx, path_data in enumerate(merged_paths):
        coords = remove_close_points(path_data['points'], min_dist=0.8)
        coords_np = np.array(coords)
        if len(coords_np) > 0: all_x.extend(coords_np[:, 0]); all_y.extend(coords_np[:, 1])
        color = colors[idx]
        short_name = get_short_name(path_data['id'])

        color = colors[idx]
        short_name = get_short_name(path_data['id'])

        # [FIX] Extract SUMO Link Index for JSON Export
        # Prioritize logic_idx from SumoInternalParser (via connection map)
        logic_idx = path_data.get('logic_idx', -1)

        if logic_idx == -1:
            try:
                # Fallback: Parse ID if logic_idx not found
                # Logic: ID format is usually ":JID_INDEX_LANE" or ":JID_INDEX_LANE + ..."
                # Example: ":R1C4_C_0_0" -> Index 0
                raw_id = path_data['id'].split(" + ")[0]
                if raw_id.startswith(":"): raw_id = raw_id[1:]
                parts = raw_id.split("_")
                if len(parts) >= 2:
                    logic_idx = int(parts[-2])
            except Exception as e:
                print(f"Warning: Could not parse logic index from {path_data['id']}: {e}")
                logic_idx = idx # Last resort fallback

        res = {'id': short_name, 'logic_idx': logic_idx, 'color': color, 'x_smooth': [], 'y_smooth'
    : [],
               'radius_data': None, 'line_obj': None, 'min_r': float('inf'), 'start_vec': None,
               'end_vec': None}
        is_spline = False
        if len(coords_np) > 2:
            try:
                deg = 3 if len(coords_np) > 3 else 2
                tck, u = splprep([coords_np[:, 0], coords_np[:, 1]], s=0.014, k=deg)
                u_new = np.linspace(0, 1, 400)
                x_s, y_s = splev(u_new, tck)
                res['x_smooth'] = x_s;
                res['y_smooth'] = y_s
                res['line_obj'] = LineString(list(zip(x_s, y_s)))
                is_spline = True
                points_smooth = np.column_stack((x_s, y_s))
                step = 5;
                min_r = float('inf');
                best_idx = -1
                for k in range(step, len(points_smooth) - step):
                    p1 = points_smooth[k - step];
                    p2 = points_smooth[k];
                    p3 = points_smooth[k + step]
                    r_geo = get_geometric_radius(p1, p2, p3)
                    if 2.0 < r_geo < 150.0:
                        if r_geo < min_r: min_r = r_geo; best_idx = k
                res['min_r'] = min_r
                if best_idx != -1:
                    px, py = points_smooth[best_idx]
                    p1 = points_smooth[best_idx - step];
                    p3 = points_smooth[best_idx + step]
                    dx = p3[0] - p1[0];
                    dy = p3[1] - p1[1];
                    l = np.sqrt(dx * dx + dy * dy)
                    if l > 0:
                        v1x, v1y = p2[0] - p1[0], p2[1] - p1[1];
                        v2x, v2y = p3[0] - p2[0], p3[1] - p2[1]
```

```
                        cross = v1x * v2y - v1y * v2x
                        sign = -1 if cross > 0 else 1
                        nx = -dy * sign / l;
                        ny = dx * sign / l
                        res['radius_data'] = (min_r, px, py, nx, ny)
                if len(points_smooth) > 20:
                    res['start_vec'] = (points_smooth[0], points_smooth[10])
                    res['end_vec'] = (points_smooth[-10], points_smooth[-1])
            except:
                pass
        if not is_spline:
            res['x_smooth'] = coords_np[:, 0];
            res['y_smooth'] = coords_np[:, 1]
            res['line_obj'] = LineString(coords)
            if len(coords) >= 2:
                res['start_vec'] = (coords[0], coords[1]);
                res['end_vec'] = (coords[-2], coords[-1])
        lines_data.append(res)
    matrix_res = [];
    intersections = [];
    n = len(lines_data)
    for i in range(n):
        row = [];
        l1 = lines_data[i]['line_obj'];
        r1 = lines_data[i]['min_r']
        for j in range(n):
            if i == j:
                row.append(None)
            else:
                sx1, sy1 = lines_data[i]['x_smooth'][0], lines_data[i]['y_smooth'][0]
                sx2, sy2 = lines_data[j]['x_smooth'][0], lines_data[j]['y_smooth'][0]
                dist_start = math.hypot(sx1 - sx2, sy1 - sy2)

                if dist_start < 5.0:
                    row.append(None)
                    continue

                l2 = lines_data[j]['line_obj'];
                r2 = lines_data[j]['min_r'];
                val = None
                if l1.intersects(l2):
                    intersect = l1.intersection(l2)
                    found_pts = []
                    if isinstance(intersect, Point):
                        found_pts.append(intersect)
                    elif isinstance(intersect, MultiPoint):
                        found_pts.extend(intersect.geoms)
                    valid_pts = []
                    for p in found_pts:
                        if l1.project(p) > 0.2: valid_pts.append(p)
                    if valid_pts:
                        valid_pts.sort(key=lambda p: l1.project(p))
                        unique_pts = [valid_pts[0]]
                        for k in range(1, len(valid_pts)):
                            if valid_pts[k].distance(unique_pts[-1]) > 1.0: unique_pts.append(
                                valid_pts[k])
                        dist_c = l1.project(unique_pts[0]);
                        dist_e = l2.project(unique_pts[0])
                        K_val = calculate_intergreen_time(dist_c, dist_e, r1, r2)
                        val = (K_val, unique_pts[0].x, unique_pts[0].y)
                        for pt in unique_pts:
                            d_c = l1.project(pt);
                            d_e = l2.project(pt)
                            k_pt = calculate_intergreen_time(d_c, d_e, r1, r2)
                            intersections.append(
```

```
                                {'pos': (pt.x, pt.y), 'val': k_pt, 'idx_i': i, 'idx_j': j})
                row.append(val)
        matrix_res.append(row)
    return {'lines': lines_data, 'matrix': matrix_res, 'intersections': intersections,
            'bounds': (all_x, all_y), 'jdata': jdata}
```

A.14. Közbensőidő-számítás: hálózat beolvasása: `sumo_parser.py`

```
import xml.etree.ElementTree as ET

class SumoInternalParser:
    def __init__(self, file_path):
        print(f"Fájl feldolgozása: {file_path}")
        self.file_path = file_path
        self.tree = ET.parse(file_path)
        self.root = self.tree.getroot()
        self.internal_lanes = {}
        self.normal_lanes = {}
        self.lane_to_link_index = {} # [NEW] Map internal lane ID -> Link Index
        self._load_data()
        self.junctions = self._group_by_junctions()
        print(f"Betöltve: {len(self.junctions)} csomópont.")

    def _load_data(self):
        # 1. Edges & Lanes
        for edge in self.root.findall("edge"):
            func = edge.get("function")
            target_dict = self.internal_lanes if func == "internal" else self.normal_lanes
            for lane in edge.findall("lane"):
                lid = lane.get("id")
                shape = lane.get("shape")
                if lid and shape:
                    points = []
                    for pair in shape.split(" "):
                        if "," in pair:
                            x, y = map(float, pair.split(","))
                            points.append((x, y))
                    target_dict[lid] = points

        # 2. [NEW] Connections (Map 'via' lane to 'linkIndex')
        for conn in self.root.findall("connection"):
            via = conn.get("via")
            link_idx = conn.get("linkIndex")
            if via and link_idx is not None:
                try:
                    self.lane_to_link_index[via] = int(link_idx)
                except ValueError:
                    pass

    def _group_by_junctions(self):
        junctions_data = []
        for junc in self.root.findall("junction"):
            if junc.get("type") in ['traffic_light', 'traffic_light_right_on_red']:
                jx = float(junc.get("x"))
                jy = float(junc.get("y"))
                jid = junc.get("id")
                shape_str = junc.get("shape")
                rel_shape = []
                if shape_str:
                    for pair in shape_str.split(" "):
                        if "," in pair:
                            sx, sy = map(float, pair.split(","))
                            rel_shape.append((sx - jx, sy - jy))
```

```
                inc_lanes_str = junc.get("incLanes")
                incoming_shapes = []
                if inc_lanes_str:
                    for lane_id in inc_lanes_str.split(" "):
                        if lane_id in self.normal_lanes:
                            points = self.normal_lanes[lane_id]
                            rel_points = [(p[0] - jx, p[1] - jy) for p in points]
                            incoming_shapes.append(rel_points)
                prefix = f":{jid}_"
                segments = []
                for lid, points in self.internal_lanes.items():
                    if lid.startswith(prefix):
                        rel_points = [(p[0] - jx, p[1] - jy) for p in points]
                        # [NEW] Attach Logic Index
                        l_idx = self.lane_to_link_index.get(lid, -1)
                        # Fallback for old parsing logic if not found (though unlikely for valid TLS)
                        segments.append({'id': lid, 'points': rel_points, 'logic_idx': l_idx})
                if segments:
                    junctions_data.append(
                        {'id': jid, 'x': jx, 'y': jy, 'segments': segments, 'shape': rel_shape,
                         'incoming_lanes': incoming_shapes})
        return junctions_data
```

A.15. Demó: fix idejű program: examples/sumo_morl/00_demo_fix_ido.py

```
"""
00 - DEMÓ: klasszikus, fix idejű jelzőlámpa-program.

Ez a legegyszerűbb vezérlés: a fázisok mindig ugyanabban a sorrendben,
mindig ugyanannyi ideig kapnak zöldet, a forgalomtól függetlenül. A
fázisváltásokat (sárga, piros, piros-sárga) a környezet a közbenső idők
alapján automatikusan beilleszti.

A demóval megnézhető, hogyan működik a szimuláció, mit látnak a
hurokdetektorok, és milyen jutalmakat ad a környezet - még mielőtt bármilyen
tanuló ágenst írnánk.

Futtatás (a repó gyökeréből):
    python examples/sumo_morl/00_demo_fix_ido.py --gui                # SUMO grafikus felülettel
    python examples/sumo_morl/00_demo_fix_ido.py                      # csak konzol
    python examples/sumo_morl/00_demo_fix_ido.py --green 40 10 20     # más zöldidők
"""
import argparse

import matplotlib

matplotlib.use("Agg")

import numpy as np

from sumo_morl import Demand, TJunctionMOEnv
from sumo_morl.visualize import plot_signal_timeline

parser = argparse.ArgumentParser(description="Fix idejű jelzőlámpa-program demó")
parser.add_argument("--gui", action="store_true", help="sumo-gui megjelenítés")
parser.add_argument("--green", type=int, nargs=3, default=[30, 10, 15], metavar=("Z0", "Z1", "Z2"),
                    help="zöldidők fázisonként [s] (alapértelmezés: 30 10 15)")
parser.add_argument("--duration", type=int, default=900, help="szimulált idő [s]")
parser.add_argument("--main", type=float, default=550, help="főirány forgalma [jármű/óra/irány]")
parser.add_argument("--side", type=float, default=250, help="mellékág forgalma [jármű/óra]")
args = parser.parse_args()

```

```python
# decision_interval=1 és min_green=1: a környezet másodpercenként kérdez, így a
# zöldidők pontosan betarthatók (a fix program maga dönt a váltásról).
env = TJunctionMOEnv(
    objectives=("queue", "fairness", "switches"),
    demand=Demand.constant(main=args.main, side=args.side),
    episode_length=args.duration,
    decision_interval=1,
    min_green=1,
    gui=args.gui,
    record_signals=True,
)
plan = env.plan
green = args.green

cycle = sum(green) + sum(plan.transition_duration(p, (p + 1) % plan.n_phases) for p in range(plan.
    n_phases))
print("=" * 72)
print(" FIX IDEJŰ PROGRAM")
print("=" * 72)
for p, name in enumerate(env.phase_names):
    nxt = (p + 1) % plan.n_phases
    print(f"  Fázis {p}: {green[p]:>3} s zöld   {plan.green_state(p)}   {name}")
    print(f"           átmenet {p}->{nxt}: {plan.transition_duration(p, nxt)} s  "
          + " -> ".join(f"{s}({d}s)" for s, d in plan.transition(p, nxt)))
print(f"  Ciklusidő: {cycle} s")
print(f"  Forgalom:  {env.demand_spec}")
print("-" * 72)
print(f"{'idő':>6}  {'esemény':<34} {'becsült jármű (W/E/S)':>22}")

obs, info = env.reset(seed=0)
total = np.zeros(env.reward_dim)
done = False
while not done:
    p = info["phase"]
    action = (p + 1) % plan.n_phases if info["time_in_phase"] >= green[p] else p
    obs, reward, terminated, truncated, info = env.step(action)
    total += reward
    done = terminated or truncated
    if info["phase_changed"]:
        veh = info["detectors"].approach_vehicles
        print(f"{info['time']:>5}s  váltás -> fázis {info['phase']} ({plan.green_state(info['phase
    '])})"
              f"{veh['W']:>12.0f} /{veh['E']:>3.0f} /{veh['S']:>3.0f}")

gt = info["ground_truth"]
print("-" * 72)
print("Epizód összjutalma (ezt látná egy ágens):")
for n, v in zip(env.objective_names, total):
    print(f"  {n:<10} {v:10.2f}")
print("Valós mutatók (csak kiértékeléshez):")
print(f"  átlagos késés járművenként: {gt['mean_delay_per_vehicle']:.1f} s")
print(f"  átlagos utazási idő:        {gt['mean_travel_time']:.1f} s")
print(f"  elindult / célba ért:       {gt['departed']} / {gt['arrived']}")

ax = plot_signal_timeline(env.signal_log, plan, t_from=1, t_to=min(2 * cycle + 1, args.duration))
ax.figure.savefig("fix_ido_jelzesterv.png", dpi=120)
print("Jelzésterv-ábra (első két ciklus) mentve: fix_ido_jelzesterv.png")
env.close()
```

A.16. Példa: ismerkedés: `examples/sumo_morl/01_ismerkedes.py`

```python
"""
01 - Ismerkedés a környezettel.
```

```
3
4  Futtatás (a repó gyökeréből):
5      python examples/sumo_morl/01_ismerkedes.py
6      python examples/sumo_morl/01_ismerkedes.py --gui     # SUMO grafikus felülettel
7  """
8  import argparse
9
10 import numpy as np
11
12 from sumo_morl import TJunctionMOEnv, list_objectives
13
14 parser = argparse.ArgumentParser()
15 parser.add_argument("--gui", action="store_true")
16 args = parser.parse_args()
17
18 env = TJunctionMOEnv(objectives=("queue", "fairness", "switches"), episode_length=600, gui=args.gui
       )
19
20 print("=== A csomópont és a jelzésterv ===")
21 print(env.scenario.describe())
22 print("\n=== Elérhető célfüggvények ===")
23 print(list_objectives())
24 print("\nAkciók (fázisok):")
25 for i, name in enumerate(env.phase_names):
26     print(f"  {i}: {name}")
27 print(f"\nMegfigyelés: {env.observation_space.shape[0]} elem, pl. {env.observation_names()[:5]} ...
       ")
28 print(f"Jutalomvektor: {env.objective_names}")
29
30 obs, info = env.reset(seed=42)
31 print(f"\nForgalmi igény ebben az epizódban: {info['demand']}")
32
33 total = np.zeros(env.reward_dim)
34 done = False
35 while not done:
36     action = env.action_space.sample()            # itt jönne az ágens döntése
37     obs, reward, terminated, truncated, info = env.step(action)
38     total += reward
39     done = terminated or truncated
40     if info["time"] % 120 < info["elapsed"]:
41         print(f"t={info['time']:4d}s  fázis={info['phase']}  jutalom={np.round(reward, 2)}  "
42               f"becsült járművek={ {k: round(v) for k, v in info['detectors'].approach_vehicles.
       items()} }")
43
44 print(f"\nEpizód összjutalma: {dict(zip(env.objective_names, np.round(total, 2).tolist()))}")
45 print(f"Valós mutatók (csak kiértékeléshez!): {info['ground_truth']}")
46 env.close()
```

A.17. Példa: baseline-ok: `examples/sumo_morl/02_baseline_osszehasonlitas.py`

```
1  """
2  02 - Klasszikus (nem tanuló) vezérlők összehasonlítása azonos forgalmon.
3
4  Ezek adják az alapvonalat (baseline), amit egy jó MORL ágensnek meg kell vernie.
5
6      python examples/sumo_morl/02_baseline_osszehasonlitas.py
7  """
8  import matplotlib
9
10 matplotlib.use("Agg")
11
12 import numpy as np
13
```

```python
from sumo_morl import (ActuatedPolicy, FixedTimePolicy, LongestQueuePolicy, RandomPolicy,
    TJunctionMOEnv,
                       evaluate_policy, plot_pareto)

env = TJunctionMOEnv(objectives=("queue", "fairness", "switches"), episode_length=1800)
seeds = [0, 1, 2]  # azonos magok = azonos forgalom minden vezérlőnek

policies = {
    "véletlen": RandomPolicy(env, seed=0),
    "fix 30/10/15": FixedTimePolicy(env, [30, 10, 15]),
    "fix 20/5/10": FixedTimePolicy(env, [20, 5, 10]),
    "járműkövető": ActuatedPolicy(env, max_green=45, gap=3),
    "leghosszabb sor": LongestQueuePolicy(env),
}

names, means = [], []
print(f"{'vezérlő':<16}" + "".join(f"{n:>12}" for n in env.objective_names) + f"{'átl. késés [s
    ]':>16}")
for name, pol in policies.items():
    res = evaluate_policy(env, pol, seeds=seeds)
    names.append(name)
    means.append(res["mean"])
    print(f"{name:<16}" + "".join(f"{v:12.1f}" for v in res["mean"])
          + f"{res['ground_truth']['mean_delay_per_vehicle']:16.1f}")
env.close()

fig = plot_pareto(np.array(means), env.objective_names, labels=names, title="Baseline vezérlők")
fig.savefig("baseline_pareto.png", dpi=120)
print("Ábra mentve: baseline_pareto.png")
```

A.18. Példa: súlyozott tanítás (SB3): `examples/sumo_morl/03_sb3_sulyozott_tanitas.py`

```python
"""
03 - MORL "külső hurok" lineáris skalarizálással (Stable-Baselines3 DQN).

Minden súlyvektorra külön ágenst tanítunk a ``w · r`` skalár jutalommal,
majd kiértékeljük a vektor-hozamokat, és felrajzoljuk a Pareto-frontot.

Telepítés:  pip install -e ".[rl]"
Futtatás:   python examples/sumo_morl/03_sb3_sulyozott_tanitas.py --timesteps 20000

A tanítás hosszabb ideig tart; a kis timesteps csak kipróbálásra jó.
"""
import argparse

import matplotlib

matplotlib.use("Agg")

import numpy as np
from stable_baselines3 import DQN

from sumo_morl import (LinearScalarization, TJunctionMOEnv, equally_spaced_weights, evaluate_policy
    ,
                       hypervolume, pareto_mask, plot_pareto, sb3_policy)

parser = argparse.ArgumentParser()
parser.add_argument("--timesteps", type=int, default=20000)
parser.add_argument("--divisions", type=int, default=2, help="súlyrács finomsága")
args = parser.parse_args()

OBJ = ("queue", "fairness", "switches")
weights = equally_spaced_weights(len(OBJ), args.divisions)
```

```
print(f"{len(weights)} súlyvektor:\n{weights}")

eval_env = TJunctionMOEnv(objectives=OBJ, episode_length=1800)
points = []
for w in weights:
    train_env = LinearScalarization(TJunctionMOEnv(objectives=OBJ, episode_length=1800, seed=0), w)
    model = DQN("MlpPolicy", train_env, learning_rate=5e-4, buffer_size=50_000, learning_starts
    =1000,
                exploration_fraction=0.3, target_update_interval=500, verbose=0, seed=0)
    model.learn(total_timesteps=args.timesteps)
    train_env.close()
    res = evaluate_policy(eval_env, sb3_policy(model), seeds=[100, 101])
    points.append(res["mean"])
    print(f"w={np.round(w, 2)}  hozam={np.round(res['mean'], 1)}  "
          f"késés={res['ground_truth']['mean_delay_per_vehicle']:.1f} s")
eval_env.close()

points = np.array(points)
ref = points.min(axis=0) - 1.0
print(f"\nPareto-optimális súlyok: {weights[pareto_mask(points)].round(2).tolist()}")
print(f"Hipertérfogat (ref={ref.round(1)}): {hypervolume(points, ref):.1f}")
fig = plot_pareto(points, OBJ, labels=[str(np.round(w, 2)) for w in weights], title="DQN, lineáris
    skalarizálás")
fig.savefig("dqn_pareto.png", dpi=120)
print("Ábra mentve: dqn_pareto.png")
```

A.19. Példa: saját célfüggvény: `examples/sumo_morl/04_sajat_celfuggveny.py`

```
"""
04 - Saját célfüggvény (jutalomkomponens) definiálása.

A célfüggvény csak hurokdetektor-adatokból (``snap``) és a vezérlő által ismert
adatokból (``ctx``) számolhat. Konvenció: nagyobb = jobb.

    python examples/sumo_morl/04_sajat_celfuggveny.py
"""
import numpy as np

from sumo_morl import FixedTimePolicy, TJunctionMOEnv, register_objective, run_episode


@register_objective("side_road_queue", scale=0.02,
                    doc="-(a déli mellékág becsült járműszáma, másodpercekre összegezve)")
def side_road_queue(snap, ctx):
    return -snap.approach_vehicle_seconds["S"]


@register_objective("main_speed", scale=0.01,
                    doc="A főirány előjelző hurkain mért átlagsebesség [m/s] * lépéshossz")
def main_speed(snap, ctx):
    mask = snap.select(kind="advance") & ~snap.select(approach="S")
    return float(snap.speed[mask].mean()) * snap.duration


env = TJunctionMOEnv(objectives=("side_road_queue", "main_speed"), episode_length=900)
for greens in ([40, 10, 8], [20, 5, 25]):
    res = run_episode(env, FixedTimePolicy(env, greens), seed=0)
    print(f"zöldidők={greens}: {dict(zip(env.objective_names, np.round(res.vector_return, 2).tolist
    ()))}")
env.close()
```

A.20. Tesztek: jelzésterv: sumo_morl/tests/test_plan.py

```python
"""A jelzésterv (fázisok, közbenső idők, átmenetek) tesztjei."""
import numpy as np
import pytest

from sumo_morl import TJunctionScenario


@pytest.fixture(scope="module")
def plan():
    return TJunctionScenario().plan


def test_groups_and_phases(plan):
    assert [g.lane_id for g in plan.groups] == ["E_C_0", "E_C_1", "S_C_0", "W_C_0"]
    assert plan.n_phases == 3
    # az összes link valamelyik fázisban zöldet kap
    served = {li for p in range(plan.n_phases) for li, ch in enumerate(plan.green_state(p)) if ch == "G"}
    assert served == set(range(plan.n_links))


def test_no_conflicting_greens(plan):
    for p in range(plan.n_phases):
        for a in plan.phases[p]:
            for b in plan.phases[p]:
                assert a == b or not plan.conflicts(a, b)


def test_intergreen_from_geometry(plan):
    # a balra kanyarodó (E_C_1) és a szemből egyenesen haladó (W_C_0) konfliktusos
    assert plan.conflicts(1, 3)
    # a keleti egyenes és a nyugati egyenes nem konfliktusos
    assert not plan.conflicts(0, 3)
    assert np.nanmin(plan.intergreen) >= 1


def _green_intervals(plan, a, b):
    """Átmenet szimulálása másodpercenként; visszaadja a jelzésképek listáját."""
    seq = [plan.green_state(a)] * 3
    for st, d in plan.transition(a, b):
        seq += [st] * int(d)
    seq += [plan.green_state(b)] * 3
    return seq


@pytest.mark.parametrize("a,b", [(a, b) for a in range(3) for b in range(3) if a != b])
def test_transition_respects_intergreen(plan, a, b):
    seq = _green_intervals(plan, a, b)
    for gi, g_from in enumerate(plan.groups):
        for gj, g_to in enumerate(plan.groups):
            k = plan.intergreen[gi, gj]
            if np.isnan(k):
                continue
            li, lj = g_from.links[0], g_to.links[0]
            end_i = max((t for t, s in enumerate(seq) if s[li] == "G" and t < len(seq) - 3), default=None)
            start_j = [t for t, s in enumerate(seq) if s[lj] == "G" and (end_i is None or t > end_i)]
            if end_i is None or not start_j:
                continue
            assert start_j[0] - end_i - 1 >= k, (a, b, gi, gj)


```

```python
@pytest.mark.parametrize("a,b", [(a, b) for a in range(3) for b in range(3) if a != b])
def test_transition_signal_sequence(plan, a, b):
    """Megszűnő zöld után sárga, induló zöld előtt piros-sárga következik."""
    seq = _green_intervals(plan, a, b)
    for li in range(plan.n_links):
        chars = "".join(s[li] for s in seq)
        assert "Gr" not in chars and "Gu" not in chars  # zöld után mindig sárga
        assert "rG" not in chars and "yG" not in chars  # zöld előtt mindig piros-sárga
```

A.21. Tesztek: környezet: sumo_morl/tests/test_env.py

```python
"""A környezet és a segédfüggvények tesztjei (rövid SUMO futásokkal)."""
import numpy as np
import pytest

from sumo_morl import (ActuatedPolicy, Demand, FixedTimePolicy, LinearScalarization,
    LongestQueuePolicy,
                       RandomPolicy, TJunctionMOEnv, WeightConditioned, equally_spaced_weights,
                       evaluate_policy, hypervolume, pareto_mask, run_episode)


@pytest.fixture(scope="module")
def env():
    e = TJunctionMOEnv(episode_length=300, seed=0)
    yield e
    e.close()


def test_spaces(env):
    obs, info = env.reset(seed=1)
    assert env.observation_space.contains(obs)
    assert obs.shape == (len(env.observation_names()),)
    obs, r, term, trunc, info = env.step(1)
    assert r.shape == env.reward_space.shape == (3,)
    assert env.observation_space.contains(obs)
    assert info["phase"] == 1 and info["phase_changed"]
    assert info["elapsed"] == env.plan.transition_duration(0, 1) + env.min_green


def test_gym_checker():
    from gymnasium.utils.env_checker import check_env

    e = TJunctionMOEnv(episode_length=60, seed=0, ground_truth=False)  # a NaN-ok miatt
    try:
        check_env(e, skip_render_check=True)
    finally:
        e.close()


def test_determinism(env):
    pol = FixedTimePolicy(env, [20, 10, 10])
    r1 = run_episode(env, pol, seed=5).vector_return
    r2 = run_episode(env, pol, seed=5).vector_return
    np.testing.assert_allclose(r1, r2)


def test_policies_run(env):
    for pol in (RandomPolicy(env, 0), ActuatedPolicy(env), LongestQueuePolicy(env)):
        res = run_episode(env, pol, seed=2, record=True)
        assert res.steps > 0 and len(res.history) == res.steps
        assert res.ground_truth["departed"] > 0


```

```python
def test_detector_estimate_close_to_truth():
    e = TJunctionMOEnv(episode_length=300, seed=0, demand=Demand.constant(600, 300))
    obs, info = e.reset(seed=0)
    pol = FixedTimePolicy(e, [20, 10, 10])
    while True:
        obs, r, te, tr, info = e.step(pol(obs, info))
        c = e._conn
        for a, edge in {"W": "W_C", "E": "E_C", "S": "S_C"}.items():
            true = sum(1 for v in c.edge.getLastStepVehicleIDs(edge) if c.vehicle.getLanePosition(v
    ) > 10)
            assert abs(info["detectors"].approach_vehicles[a] - true) <= 3
        if tr:
            break
    e.close()


def test_scalarization(env):
    w = LinearScalarization(env, [1.0, 0.0, 0.0])
    w.reset(seed=0)
    _, r, _, _, info = w.step(0)
    assert isinstance(r, float) and r == pytest.approx(float(info["vector_reward"][0]))
    wc = WeightConditioned(env, seed=0)
    obs, _ = wc.reset(seed=0)
    assert obs.shape[0] == env.observation_space.shape[0] + 3


def test_evaluate(env):
    res = evaluate_policy(env, FixedTimePolicy(env, [20, 10, 10]), seeds=[0, 1])
    assert res["returns"].shape == (2, 3)


def test_pareto_and_hv():
    pts = np.array([[1, 3], [2, 2], [3, 1], [1, 1], [2, 1.5]])
    assert pareto_mask(pts).tolist() == [True, True, True, False, False]
    assert hypervolume(pts, [0, 0]) == pytest.approx(6.0)
    assert hypervolume(np.array([[1, 1, 1]]), [0, 0, 0]) == pytest.approx(1.0)
    pts3 = np.array([[1, 2, 1], [2, 1, 1]])
    assert hypervolume(pts3, [0, 0, 0]) == pytest.approx(3.0)
    w = equally_spaced_weights(3, 2)
    assert np.allclose(w.sum(axis=1), 1) and len(w) == 6
```

A.22. Telepítési leíró: `pyproject.toml`

```toml
[build-system]
requires = ["setuptools>=61"]
build-backend = "setuptools.build_meta"

[project]
name = "sumo-morl"
version = "0.1.0"
description = "Többcélú megerősítéses tanulás (MORL) jelzőlámpa-vezérléshez SUMO-ban - oktatási kö
    nyvtár"
readme = "docs/sumo_morl/README.md"
requires-python = ">=3.9"
dependencies = [
    "numpy",
    "gymnasium>=0.29",
    "eclipse-sumo>=1.18",
    "traci>=1.18",
    "sumolib>=1.18",
    "matplotlib",
    "scipy",
    "shapely>=2",
```

```
    "networkx",
]

[project.optional-dependencies]
rl = ["stable-baselines3>=2.0"]
morl = ["morl-baselines"]
dev = ["pytest"]

[tool.setuptools]
packages = ["sumo_morl"]
# A meglévő közbensőidő-számító modulok (a repó gyökerében) is települnek,
# mert a sumo_morl ezeket használja.
py-modules = ["geometry_utils", "sumo_parser"]

[tool.setuptools.package-data]
sumo_morl = ["data/*.xml"]

[tool.pytest.ini_options]
testpaths = ["sumo_morl/tests"]
```